LECTURE 20

Decision Trees

An alternate technique for classification.

Presented by Dr . Devan Becker, Data200

Adapted from Data 100, Summer 2023 @ UC Berkeley by Bella Crouch and Dominic Liu

Content credit: Acknowledgments

CC BY NC SA

Today's Roadmap

Multiclass Classification

Decision Trees (conceptually)

- Creating Decision Trees in `sklearn`
- Evaluating Tree Accuracy
- Overfit Decision Tree Example

The Decision Tree Generation Algorithm

- Evaluating Split Quality
- Entropy
- Generating Trees Using Entropy

Avoiding Overfitting

- Restricting Decision Tree Complexity
- Random Forests

Summary and Context

CC BY NC SA

Multiclass Classification

Multiclass Classification

Decision Trees (conceptually)

- Creating Decision Trees in `sklearn`
- Evaluating Tree Accuracy
- Overfit Decision Tree Example

The Decision Tree Generation Algorithm

- Evaluating Split Quality
- Entropy
- Generating Trees Using Entropy

Avoiding Overfitting

- Restricting Decision Tree Complexity
- Random Forests

Summary and Context

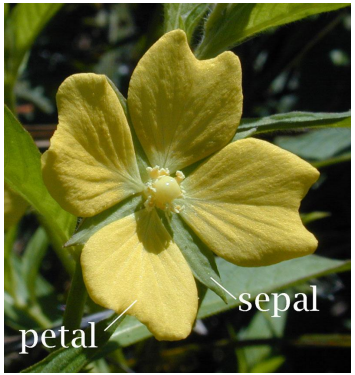
CC BY NC SA

Example: Flower Classification

The Iris flower data set is a commonly used example:

- Created by statistician/biologist Ronald Fisher for his paper “The use of multiple measurements in taxonomic problems”.
- Data set consists of 150 flower measurements from 3 different species: setosa, versicolor, and virginica.
- For each, we have “petal length”, “petal width”, “sepal length”, “sepal width”.

Goal is to predict species from other data.



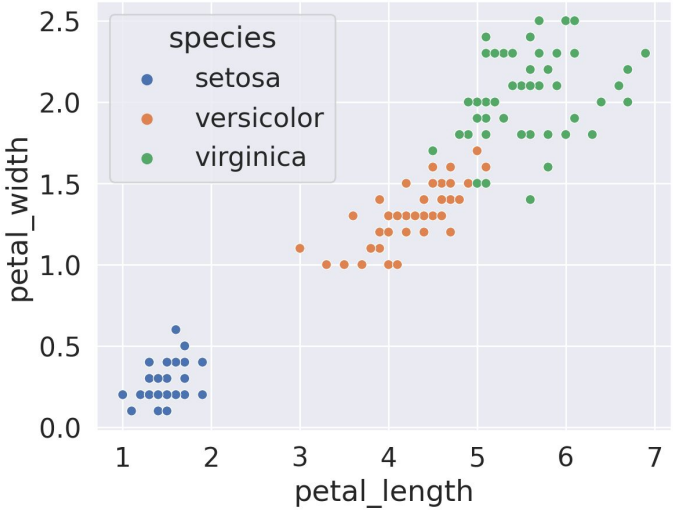
https://en.wikipedia.org/wiki/Sepal

sepal_length	sepal_width	petal_length	petal_width	species
5.8	2.6	4.0	1.2	versicolor
5.1	3.8	1.9	0.4	setosa
5.0	3.4	1.5	0.2	setosa
5.1	3.7	1.5	0.4	setosa
5.7	3.0	4.2	1.2	versicolor

CC BY NC SA

Example: Logistic Regression Using Petal Data Only

The plot below shows the width and length of the petals of each flower, with the species annotated in the form of color.



CC BY NC SA

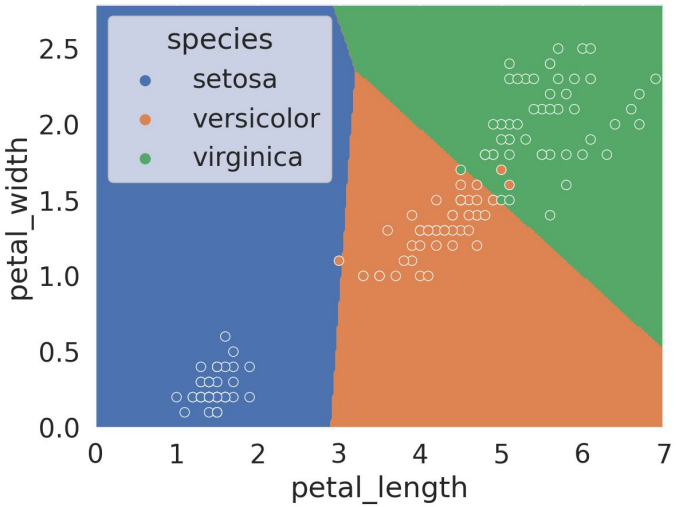
Example: Logistic Regression Using Petal Data Only

The plot below shows the width and length of the petals of each flower, with the species annotated in the form of color.

- If we train a 3 class logistic regression model in sklearn on this data, we end up with the model below.

```
from sklearn.linear_model import LogisticRegression
logistic_regression_model = LogisticRegression(multi_class = 'ovr')
logistic_regression_model = logistic_regression_model.fit(iris_data[["petal_length", "petal_width"]], iris_data["species"])
```

Because logistic regression models are linear, the decision boundaries are linear.



CC BY NC SA

One vs. Rest Classification

In our previous Logistic Regression lectures, our classifier differentiates between two classes: positive or negative. How does the model below work?

```
from sklearn.linear_model import LogisticRegression
logistic_regression_model = LogisticRegression(multi_class = 'ovr')
logistic_regression_model = logistic_regression_model.fit(iris_data[["petal_length", "petal_width"]], iris_data["species"])
```

In the sklearn code above, I set "`multi_class = 'ovr'`", which stands for "one vs. rest", meaning that sklearn actually built 3 binary classifiers:

- Classifier 1: Setosa or not.
- Classifier 2: Versicolor or not.
- Classifier 3: Virginica or not.

Output of `predict` is the highest probability of the three classifiers:

```
logistic_regression_model.predict([[1.4, 0.2]])
```

```
array(['setosa'], dtype=object)
```


CC BY NC SA

Decision Trees (conceptually)

Multiclass Classification

Decision Trees (conceptually)

- Creating Decision Trees in `sklearn`
- Evaluating Tree Accuracy
- Overfit Decision Tree Example

The Decision Tree Generation Algorithm

- Evaluating Split Quality
- Entropy
- Generating Trees Using Entropy

Avoiding Overfitting

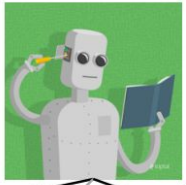
- Restricting Decision Tree Complexity
- Random Forests

Summary and Context

CC BY NC SA

Taxonomy of Machine Learning

Taxonomy of Machine Learning



Labeled Data

Reward

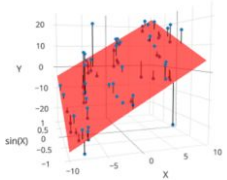
Unlabeled Data

Supervised Learning

Quantitative Response

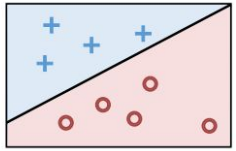
Categorical Response

Regression



OLS; linear regression
w/regularization

Classification



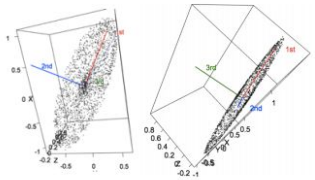
k-nearest neighbors [Data 8]
Logistic Regression
Decision Trees

Reinforcement Learning
(not covered)

Alpha Go

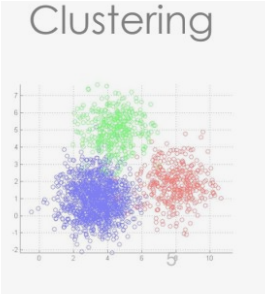
Unsupervised Learning

Dimensionality Reduction



PCA

Clustering



CC BY NC SA

Data 200: Linear Models

In Data 200, we've only used linear models.

- Linear regression: Output is the weighted sum of the features.
- Logistic regression: Output is $\boldsymbol{\sigma}$(the weighted sum of the features).

There are countless techniques for regression and classification.

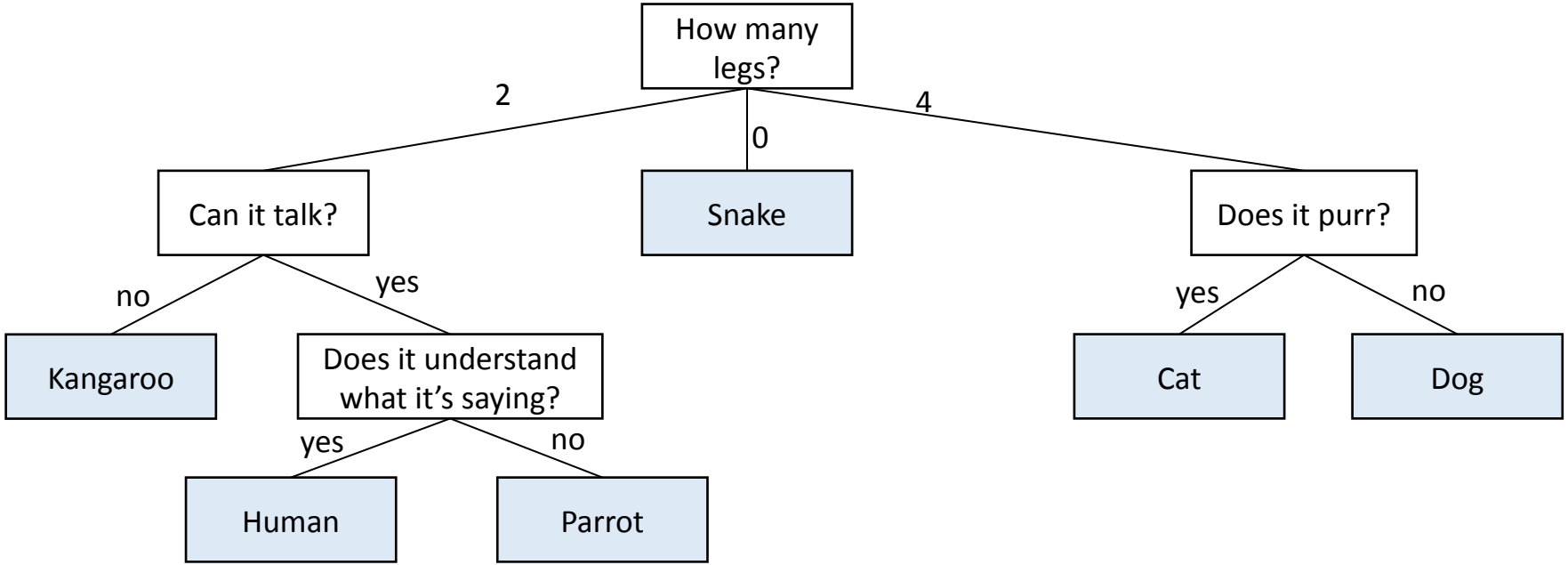
- Today we'll describe one such technique: Decision Trees (**non-linear**).
- For more such techniques see ST494

CC BY NC SA

Decision Trees

A Decision Tree is an alternate way to classify data. It is simply a tree of questions that must be answered in sequence to yield a predicted classification.

Why is it a tree?

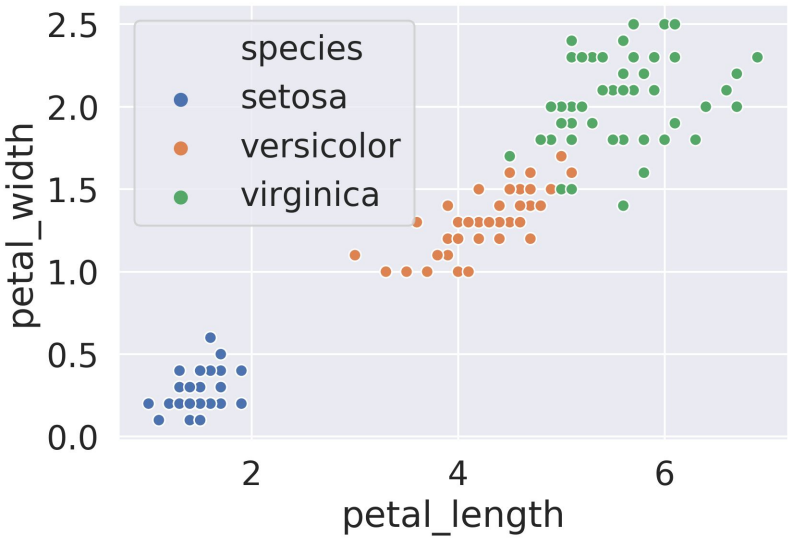


CC BY NC SA

Example: Using Petal Data Only

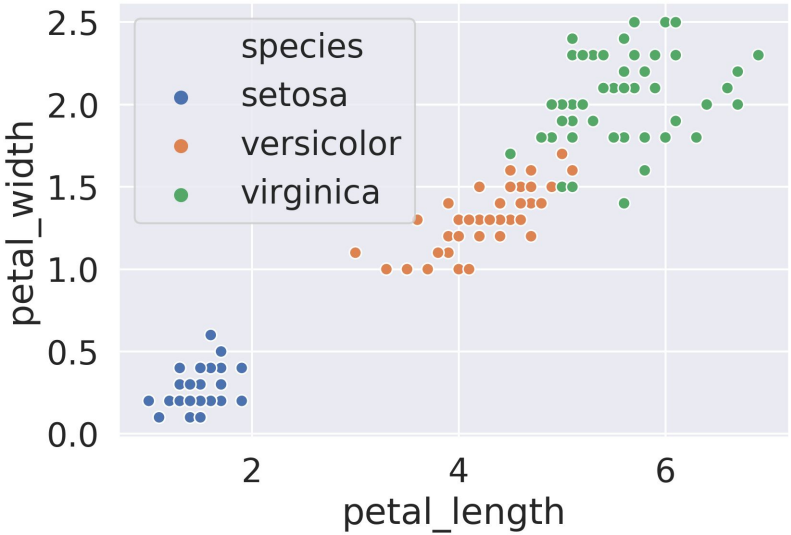
The plot below shows the width and length of the petals of each flower, with the species annotated in the form of color.

We can build a decision tree manually just by looking at this picture.



CC BY NC SA

Example: Using Petal Data Only



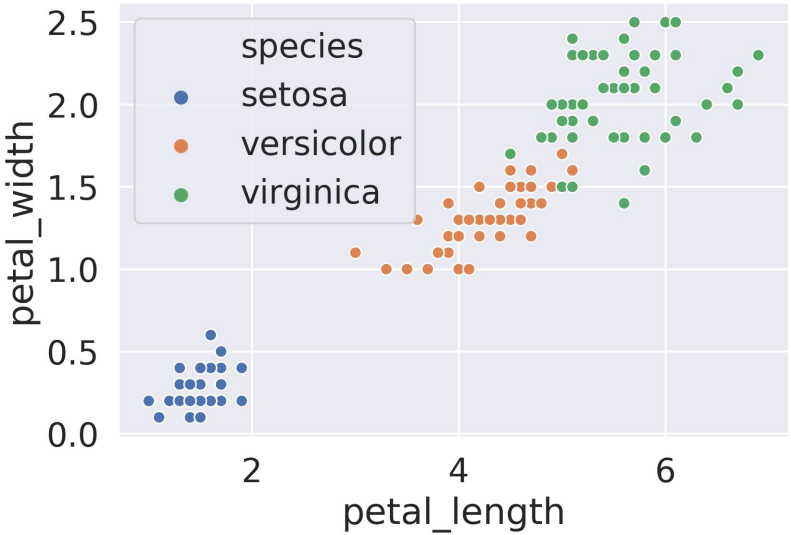
CC BY NC SA

Example: Using Petal Data Only

Is petal_width < 0.75 and petal_length < 2?

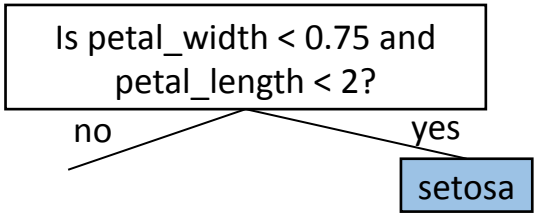
no

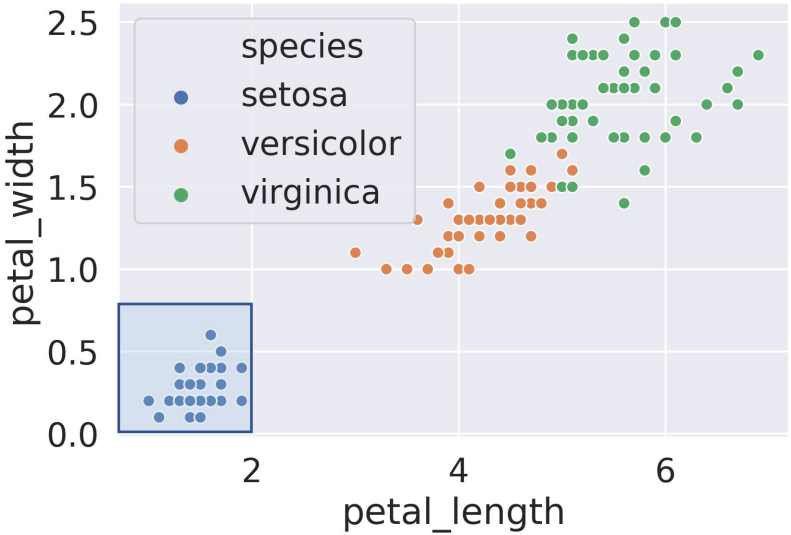
yes



CC BY NC SA

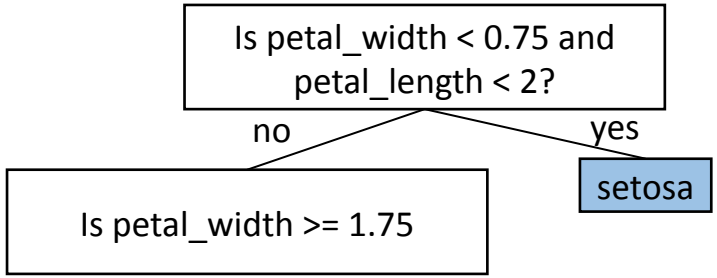
Example: Using Petal Data Only

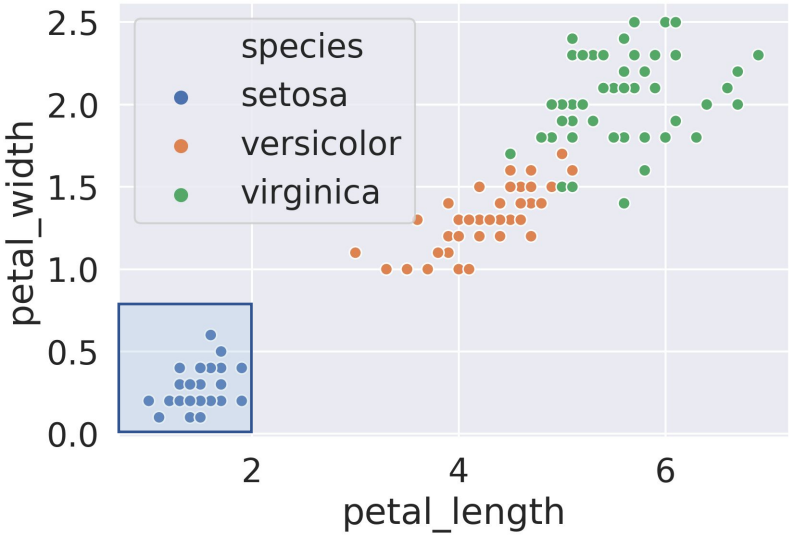




CC BY NC SA

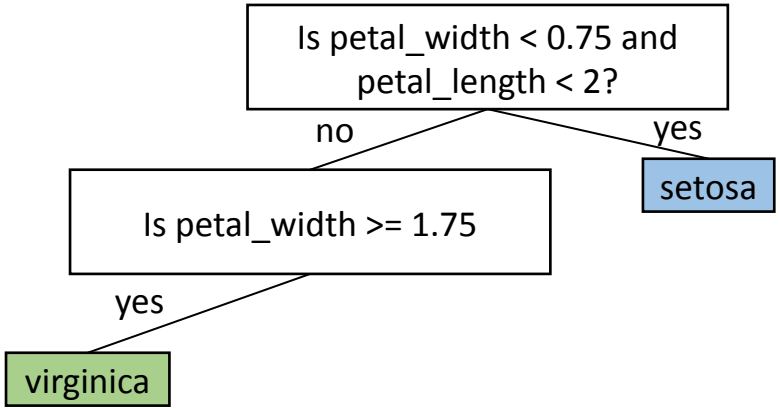
Example: Using Petal Data Only

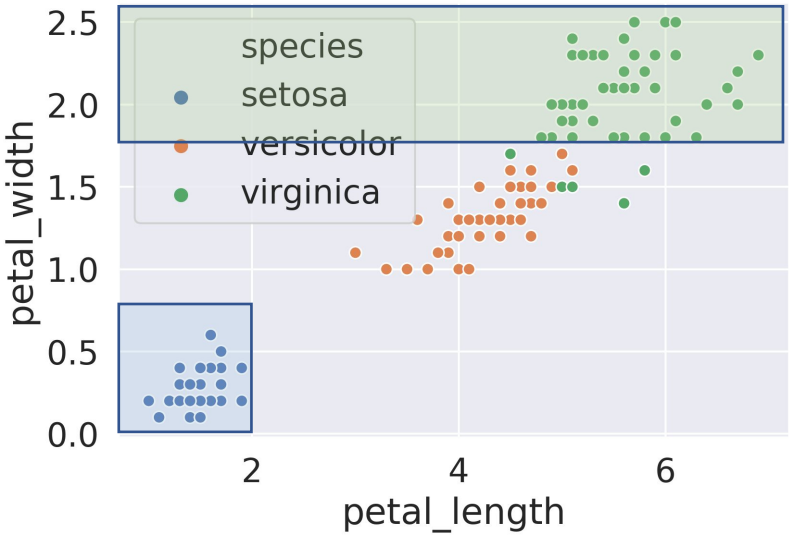




CC BY NC SA

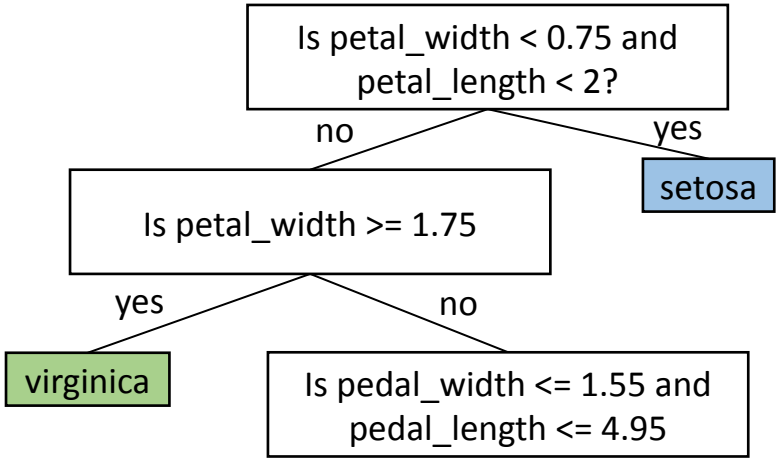
Example: Using Petal Data Only

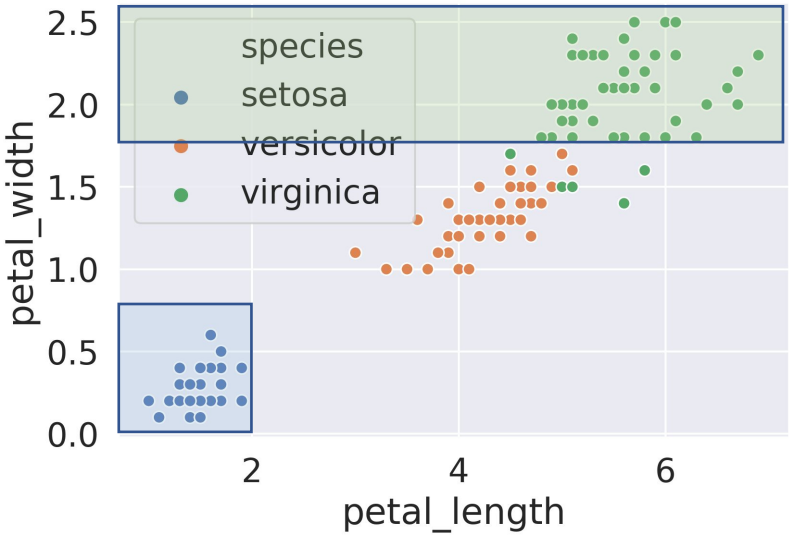




CC BY NC SA

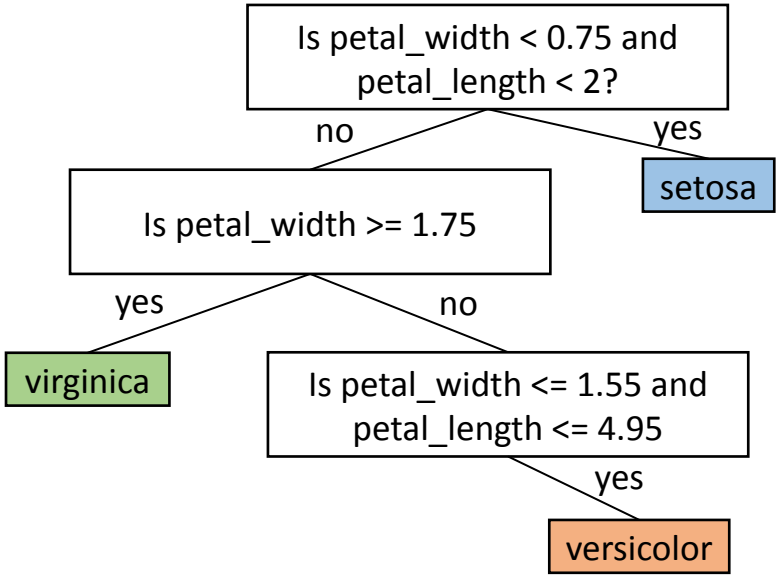
Example: Using Petal Data Only

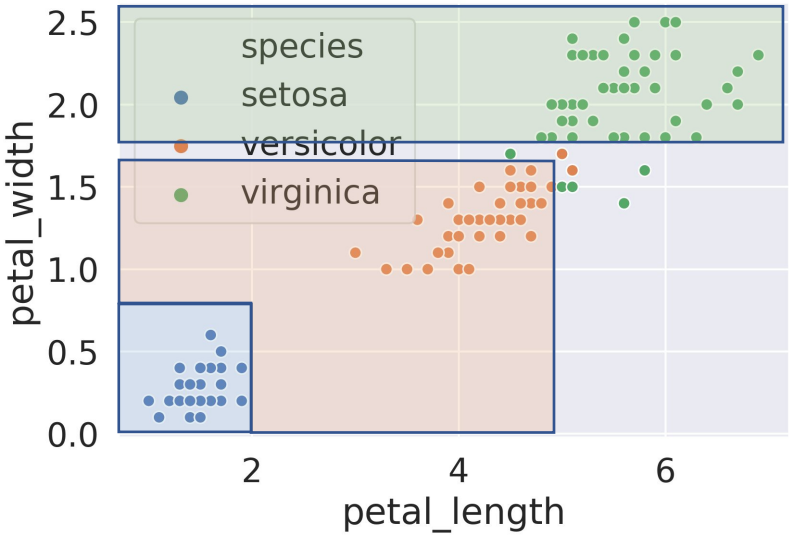




CC BY NC SA

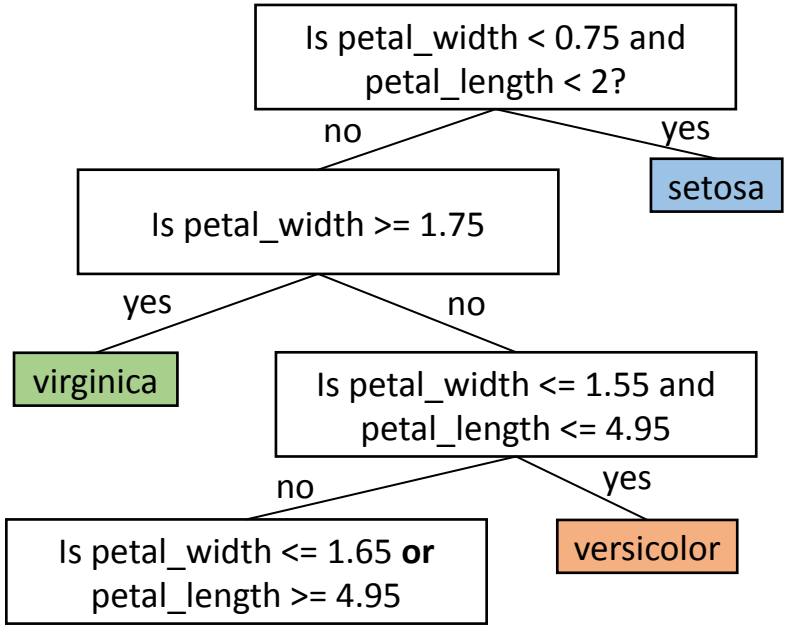
Example: Using Petal Data Only

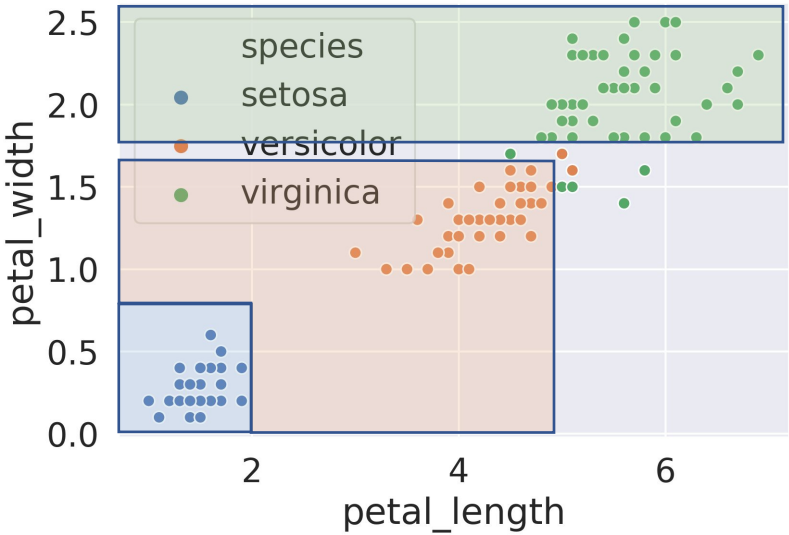




CC BY NC SA

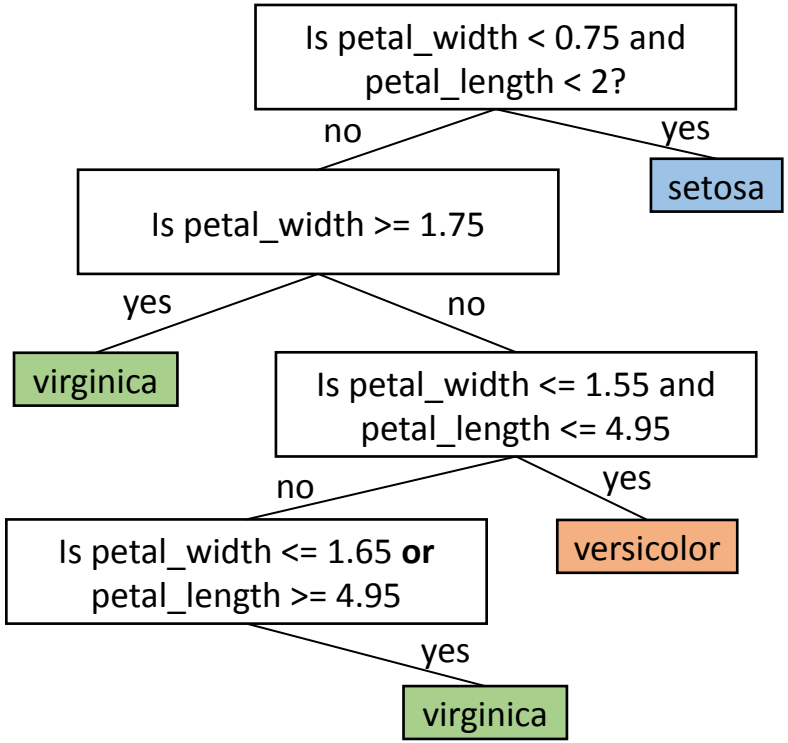
Example: Using Petal Data Only

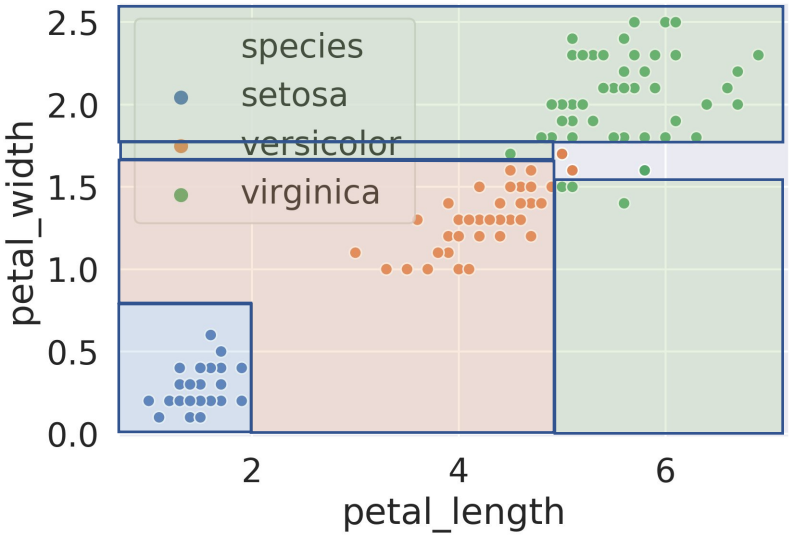




CC BY NC SA

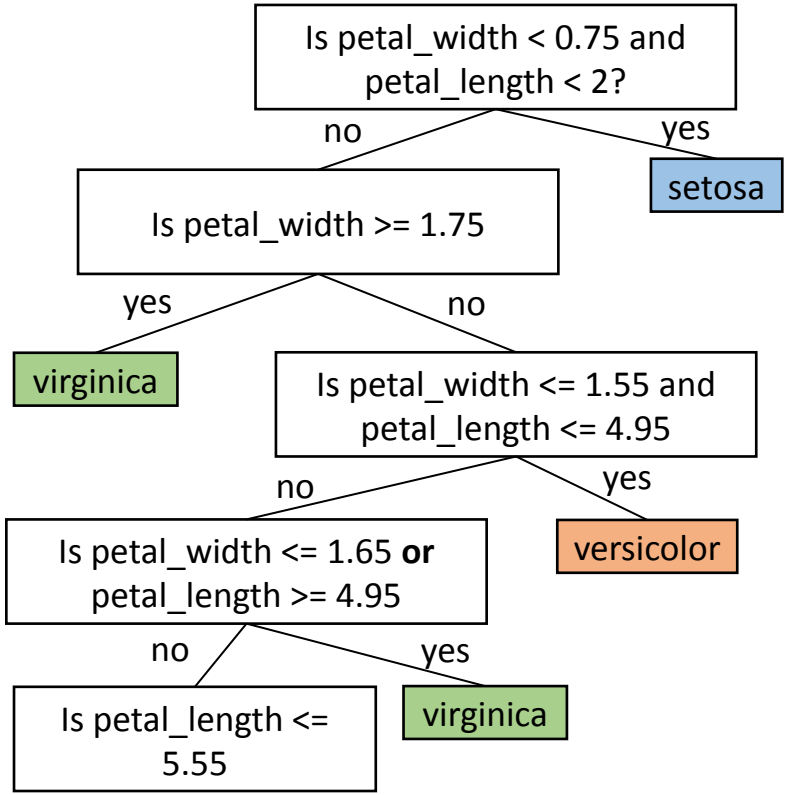
Example: Using Petal Data Only

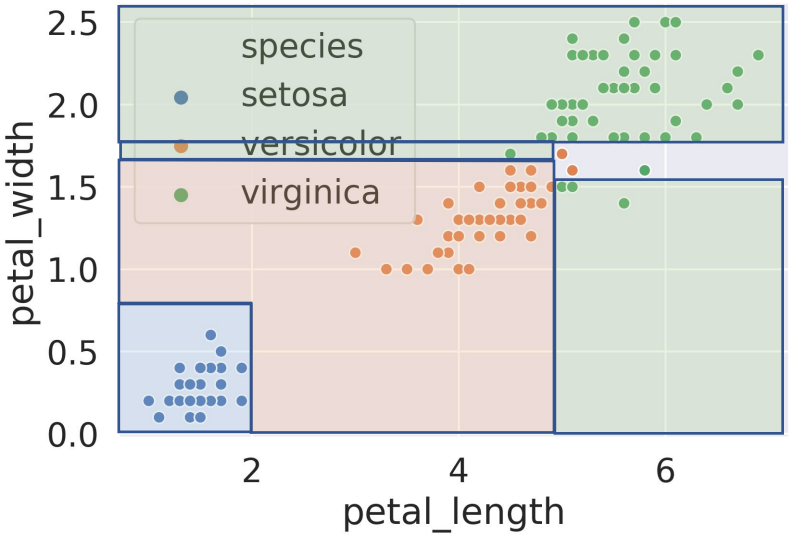




CC BY NC SA

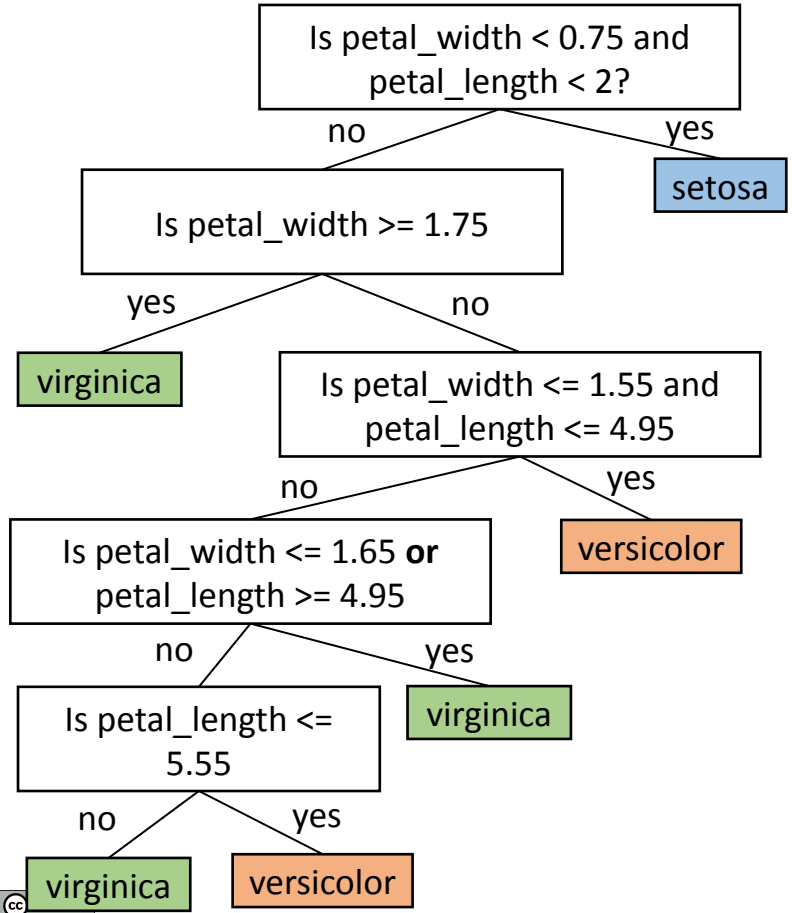
Example: Using Petal Data Only

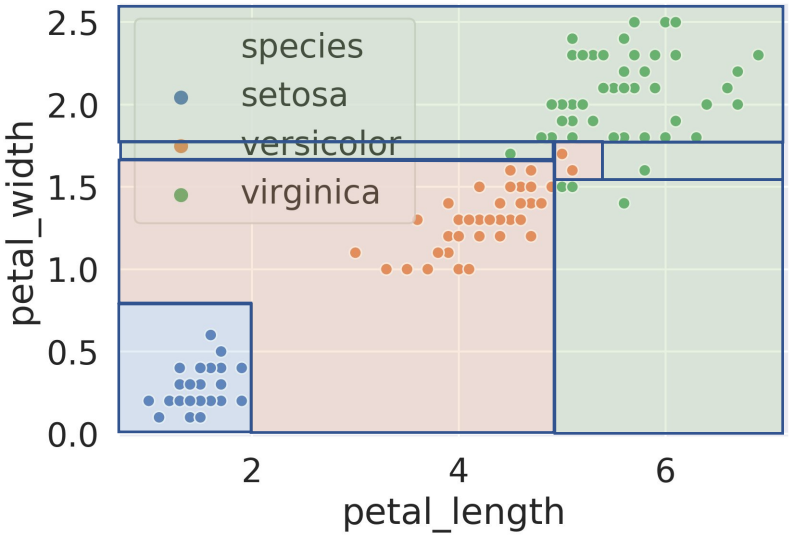




CC BY NC SA

Example: Using Petal Data Only

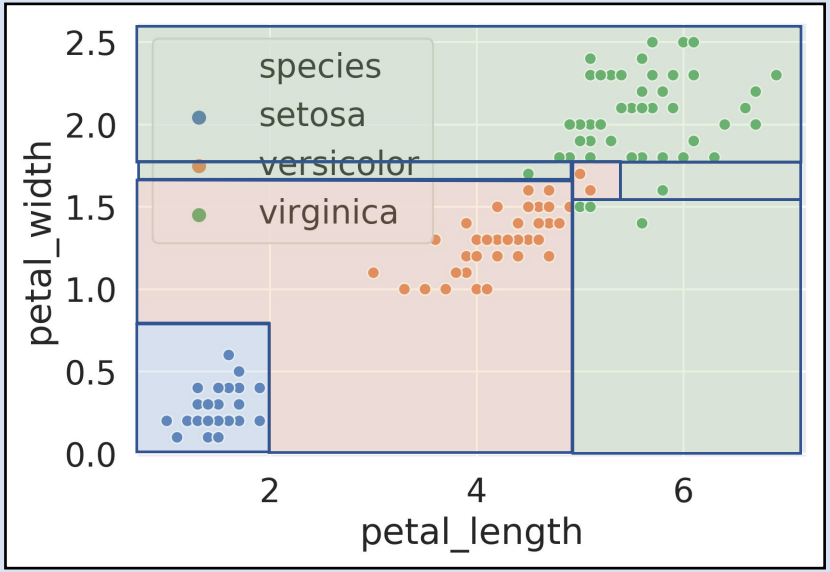




Example: Using Petal Data Only

How accurate is our decision tree model on the training data?

Is this good or bad?

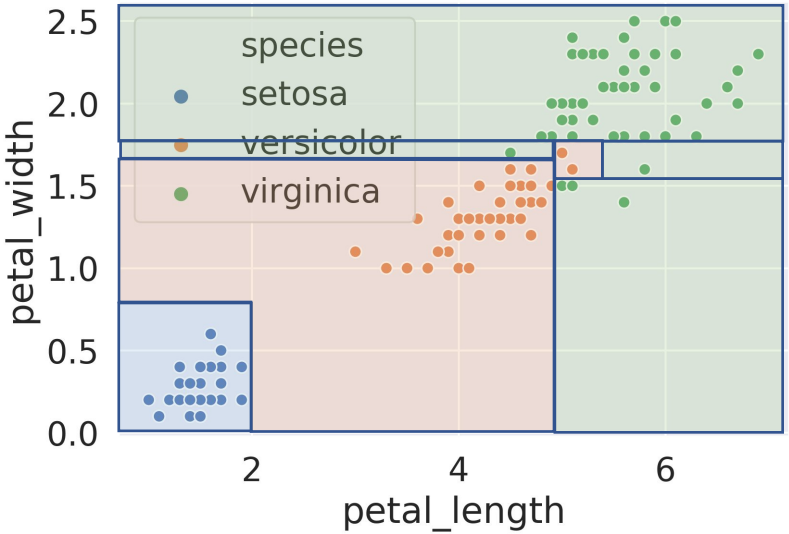


CC BY NC SA

Example: Using Petal Data Only

How accurate is our decision tree model on the training data?

- It seems like it gets every point correct*.
  - *Caveat: it actually doesn't - it's very close though!



Is this good or bad?

- I'd argue bad.
- Seems likely to result in overfitting!
- Will discuss overfitting more later.

First, let's see how we can build decision trees for classification using Scikit-learn.

CC BY NC SA

Decision Trees in `sklearn`

Multiclass Classification

Decision Trees (conceptually)

- **Creating Decision Trees in `sklearn`**
- Evaluating Tree Accuracy
- Overfit Decision Tree Example

The Decision Tree Generation Algorithm

- Evaluating Split Quality
- Entropy
- Generating Trees Using Entropy

Avoiding Overfitting

- Restricting Decision Tree Complexity
- Random Forests

Summary and Context

CC BY NC SA

Decision Tree Models With `sklearn`

The code to build a decision tree model in **`sklearn`** is very similar to what we saw for building linear and logistic regression models:

```
from sklearn import tree
decision_tree_model = tree.DecisionTreeClassifier(criterion='entropy')
decision_tree_model = decision_tree_model.fit(iris_data[["petal_length", "petal_width"]], iris_data["species"])
```


CC BY NC SA

Decision Tree Models With `sklearn`

The code to build a decision tree model in **`sklearn`** is very similar to what we saw for building linear and logistic regression models:

```
from sklearn import tree
decision_tree_model = tree.DecisionTreeClassifier(criterion='entropy')
decision_tree_model = decision_tree_model.fit(iris_data[["petal_length", "petal_width"]], iris_data["species"])
```

- Using the decision tree model:

	sepal_length	sepal_width	petal_length	petal_width	species
18	5.7	3.8	1.7	0.3	setosa
140	6.7	3.1	5.6	2.4	virginica
104	6.5	3.0	5.8	2.2	virginica
11	4.8	3.4	1.6	0.2	setosa

```
decision_tree_model.predict(four_random_rows[["petal_length", "petal_width"]])
```

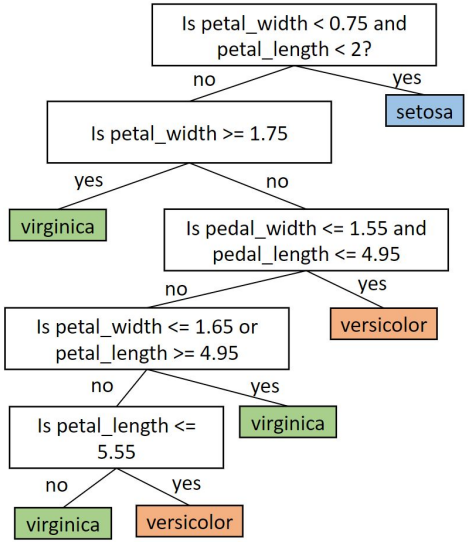
```
array(['setosa', 'virginica', 'virginica', 'setosa'], dtype=object)
```


CC BY NC SA

Visualizing Decision Tree Models

Suppose we want to visualize the decision tree, similar to what we saw earlier:

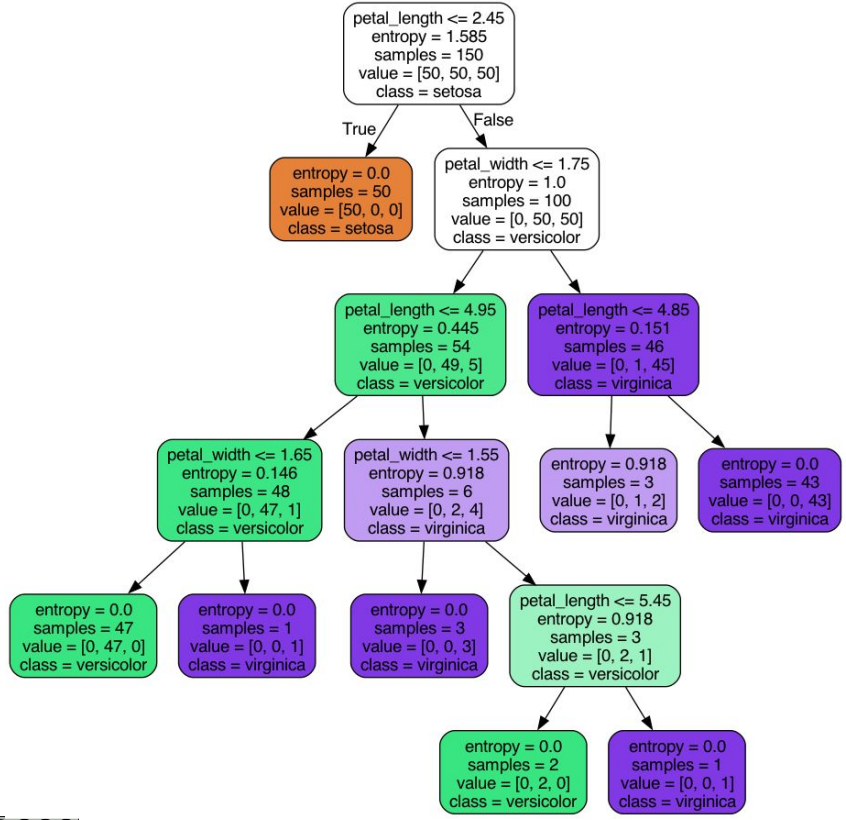
```
from sklearn import tree
decision_tree_model = tree.DecisionTreeClassifier()
decision_tree_model = decision_tree_model.fit(iris_data[["petal_length", "petal_width"]], iris_data["species"])
```



CC BY NC SA

Visualizing Decision Tree Models

Let's use **`GraphViz`** to get a nice picture of our tree!




```
import graphviz
dot_data = tree.export_graphviz(decision_tree_model, out_file=None,
                                feature_names=["petal_length", "petal_width"],
                                class_names=["setosa", "versicolor", "virginica"],
                                filled=True, rounded=True)
graph = graphviz.Source(dot_data)
graph.render(format="png", filename="iris_tree")
graph
```


CC BY NC SA

Visualizing Decision Tree Models

Let's use `GraphViz` to get a nice picture of our tree!

```
import graphviz
dot_data = tree.export_graphviz(decision_tree_model, out_file=None,
                                feature_names=["petal_length", "petal_width"],
                                class_names=["setosa", "versicolor", "virginica"],
                                filled=True, rounded=True)
graph = graphviz.Source(dot_data)
graph.render(format="png", filename="iris_tree")
graph
```

petal_length <= 2.45
entropy = 1.585
samples = 150
value = [50, 50, 50]
class = setosa

In each box, we see:

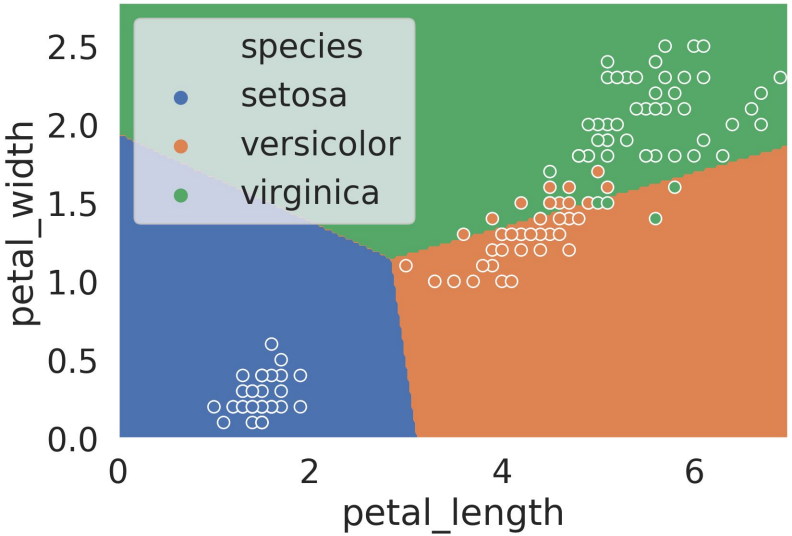
- The rule.
- The entropy at that node (more later).
- The number of samples that remain after applying all of the above rules.
- The number of samples that remain in each class.
- The most likely class.

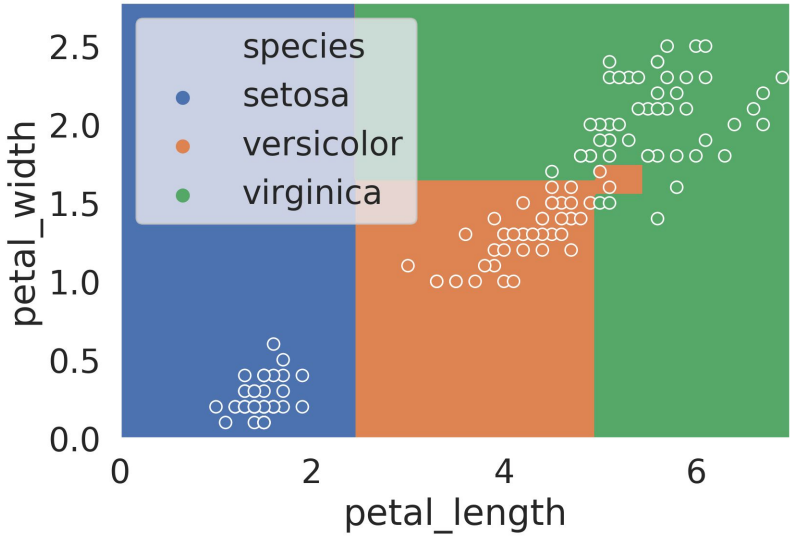
CC BY NC SA

Visualizing Decision Boundaries for Logistic Regression vs. Decision Tree

Plotting the decision boundaries for our logistic regression and decision tree models, we get the results below.

- Decision tree has nonlinear boundary, and appears to get 100% accuracy.
  - Let's calculate the exact accuracy rather than just relying on our eyes to look at a somewhat complex visualization.





CC BY NC SA

Evaluating Tree Accuracy

Multiclass Classification

Decision Trees (conceptually)

- Creating Decision Trees in `sklearn`
- **Evaluating Tree Accuracy**
- Overfit Decision Tree Example

The Decision Tree Generation Algorithm

- Evaluating Split Quality
- Entropy
- Generating Trees Using Entropy

Avoiding Overfitting

- Restricting Decision Tree Complexity
- Random Forests

Summary and Context

CC BY NC SA

Measuring the Performance of Our Model

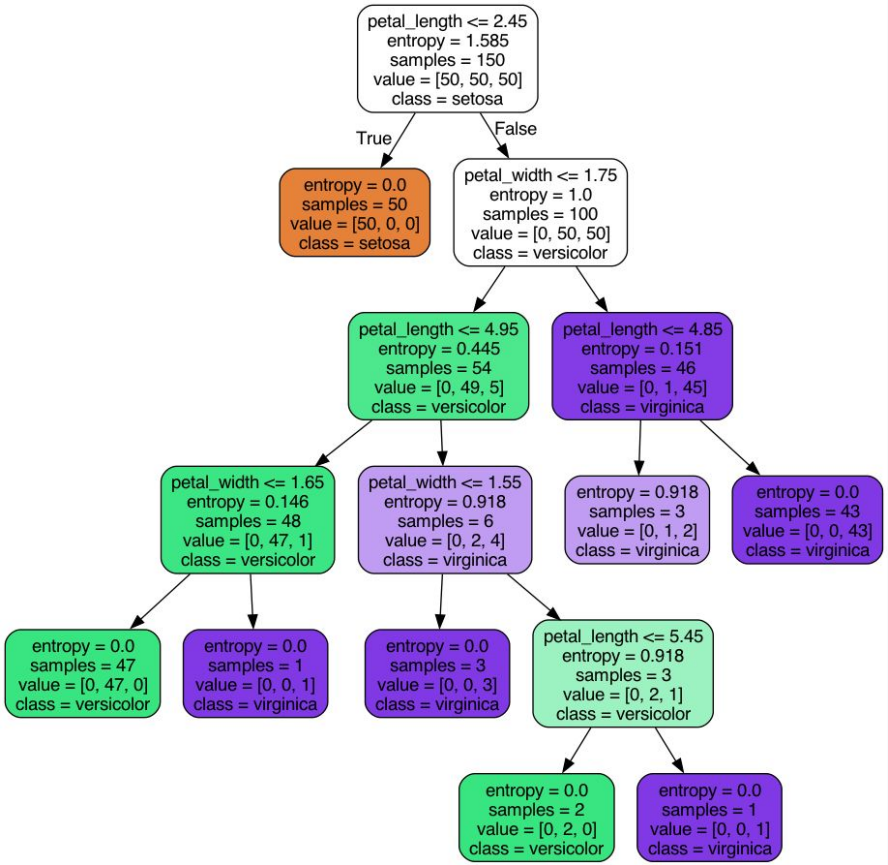
Running the code below, we see that we only get 99.3% accuracy.

```
from sklearn.metrics import accuracy_score
predictions = decision_tree_model.predict(iris_data[["petal_length", "petal_width"]])
accuracy_score(predictions, iris_data["species"])
```

To understand why, let's look back at our decision tree model.

CC BY NC SA

Understanding Our Decision Tree

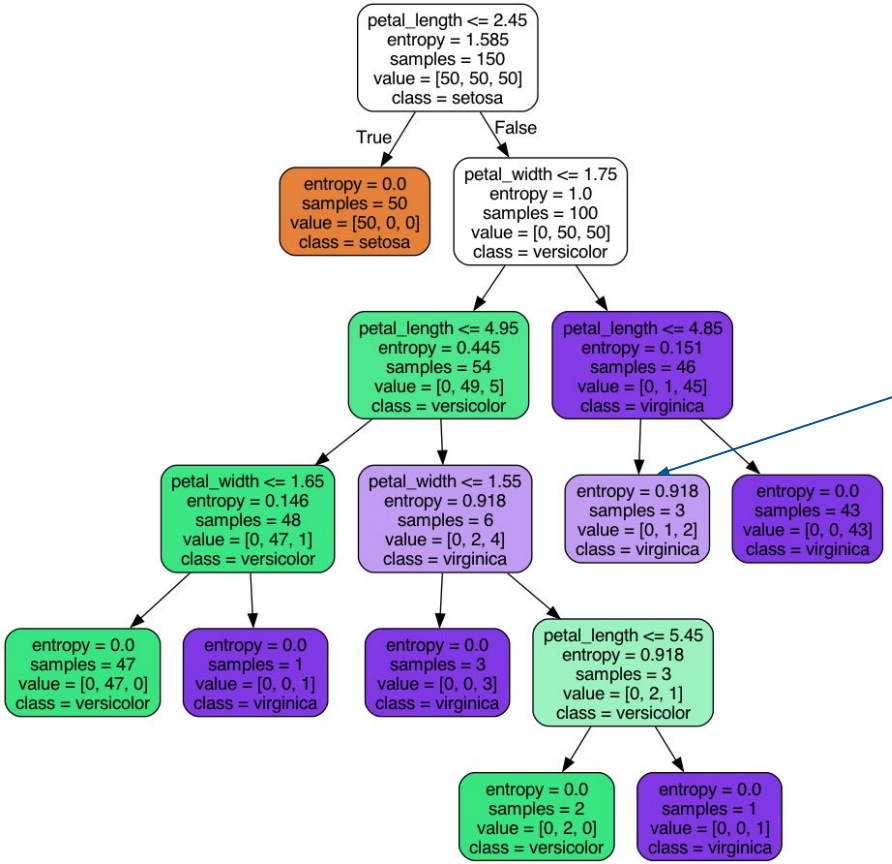


There is one terminal decision point where there is more than one possible right answer.

Can you find it?

BY NC SA

Understanding Our Decision Tree



There is one terminal decision point where there is more than one possible right answer.

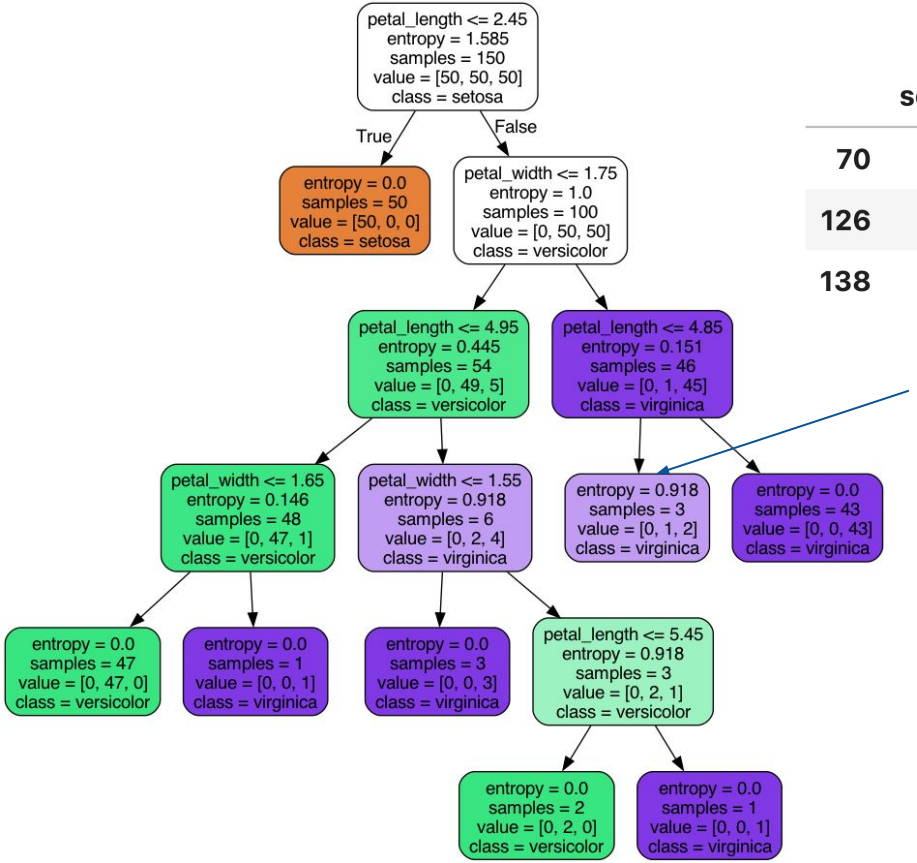
The model was unable to come up with a decision rule to resolve these last 3 samples.

Let's see why by looking at these three observations in our dataset.

CC BY NC SA

Understanding Our Decision Tree

```
iris_data[(iris_data["petal_length"]> 2.45)&(iris_data["petal_width"]> 1.75)&(iris_data["petal_length"]<=4.85)]
```



	sepal_length	sepal_width	petal_length	petal_width	species
70	5.9	3.2	4.8	1.8	versicolor
126	6.2	2.8	4.8	1.8	virginica
138	6.0	3.0	4.8	1.8	virginica

There is one terminal decision point where there is more than one possible right answer.

- In the original data set, there was a versicolor iris with the same petal measurements as two virginicas.

CC BY NC SA

Overfitting and Decision Trees

	sepal_length	sepal_width	petal_length	petal_width	species
70	5.9	3.2	4.8	1.8	versicolor
126	6.2	2.8	4.8	1.8	virginica
138	6.0	3.0	4.8	1.8	virginica

`sklearn` decision trees (and in general, all decision trees when unrestricted) will always have perfect accuracy on the training data, **EXCEPT** when there are samples from different categories with the exact same features.

- Example: If the versicolor above had a petal_length of 4.8001, we'd have 100% training accuracy.

This tendency for perfect accuracy should give us concern about overfitting.

- Model is extremely sensitive / has high **variance**.

CC BY NC SA

Overfit Decision Tree Example

Multiclass Classification

Decision Trees (conceptually)

- Creating Decision Trees in `sklearn`
- Evaluating Tree Accuracy
- **Overfit Decision Tree Example**

The Decision Tree Generation Algorithm

- Evaluating Split Quality
- Entropy
- Generating Trees Using Entropy

Avoiding Overfitting

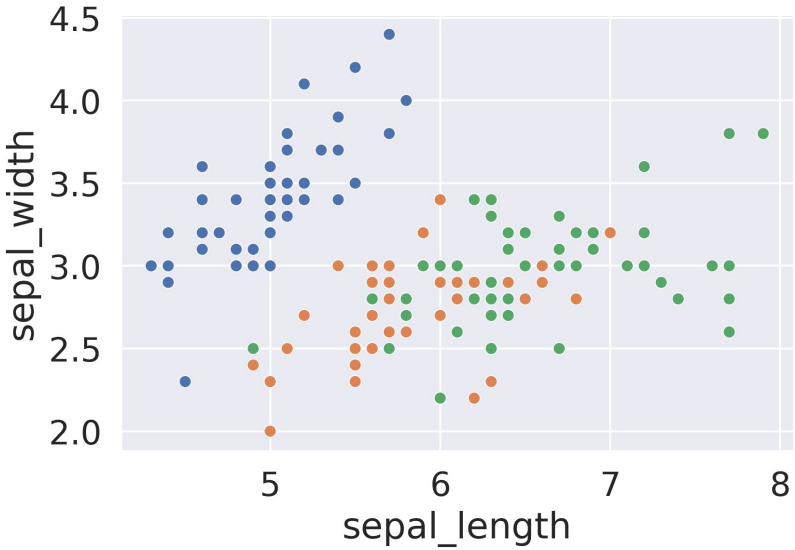
- Restricting Decision Tree Complexity
- Random Forests

Summary and Context

CC BY NC SA

Overfitting and Sepal Data

To see how bad this overitting can get, consider a scatterplot of the **“`sepal_length`”** and **“`sepal_width`”** of the same dataset as before

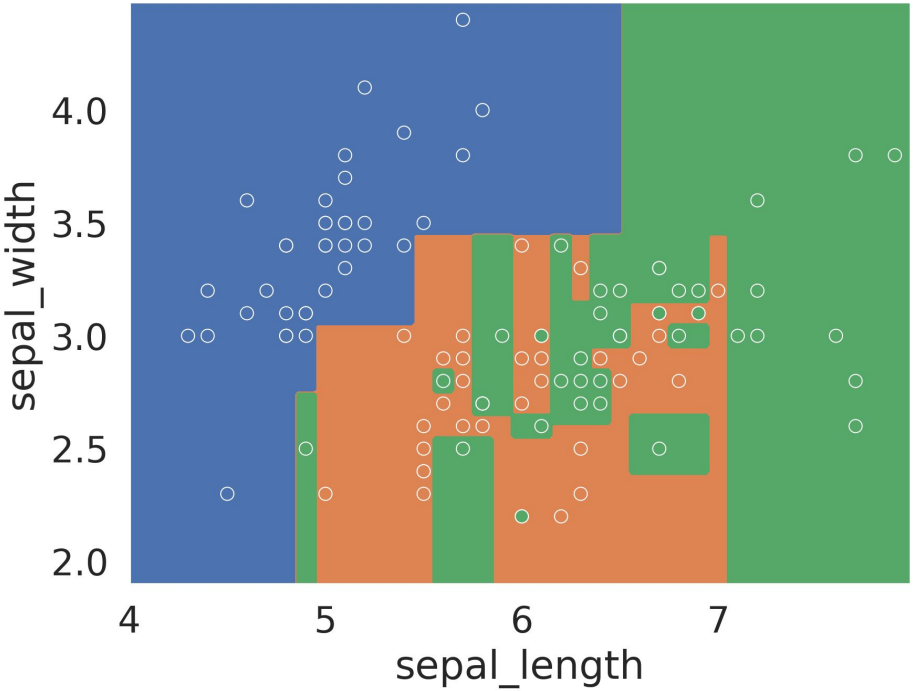


CC BY NC SA

Sepal Decision Boundaries

Fitting a decision tree to this data yields highly erratic boundaries.

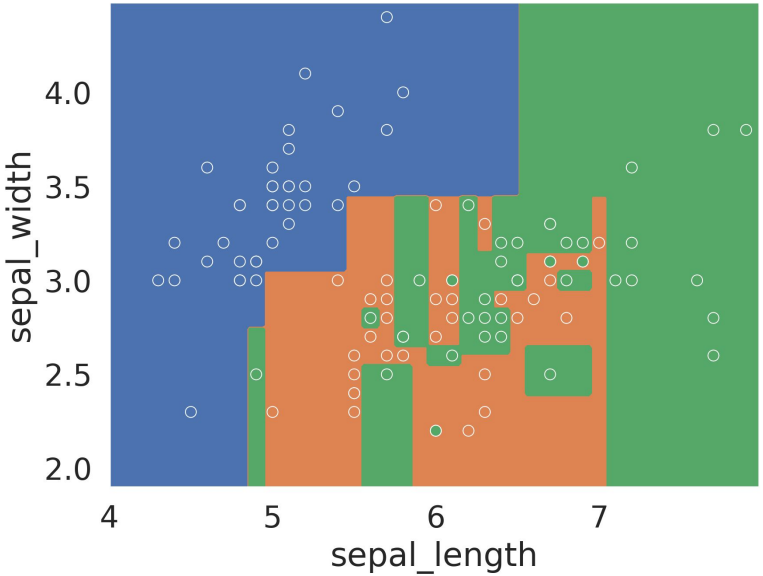
- Unlikely to reflect any sort of physical reality.
- No matter how complex the data, decisions trees will create boundaries that are arbitrarily complex to reflect the data.

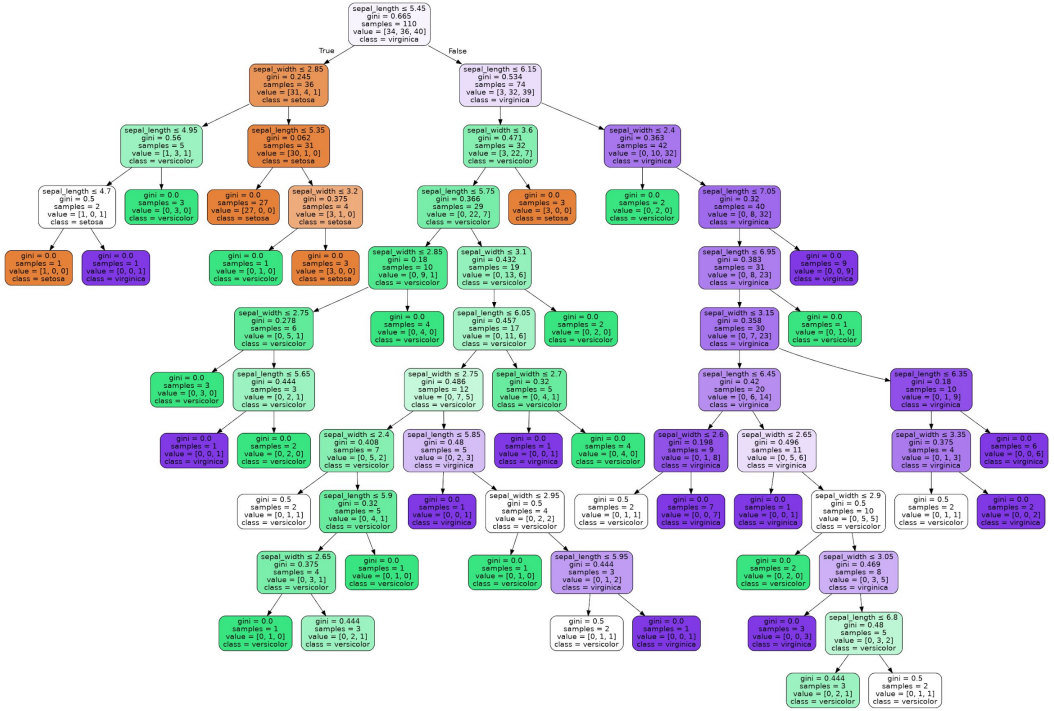


CC BY NC SA

Sepal Decision Tree

We can also visualize the decision tree rules, shown below and to the right.





CC BY NC SA

Multidimensional Decision Trees

Naturally, we can include even more features. For example, if we want to use the petal AND sepal measurements, we simply train the decision tree on all four columns of the data.

```
decision_tree_model_4d = tree.DecisionTreeClassifier(criterion="entropy")
decision_tree_model_4d = decision_tree_model_4d.fit(train_iris_data[["petal_length", "petal_width",
                                                                      "sepal_length", "sepal_width"]], train_iris_data["species"])
```

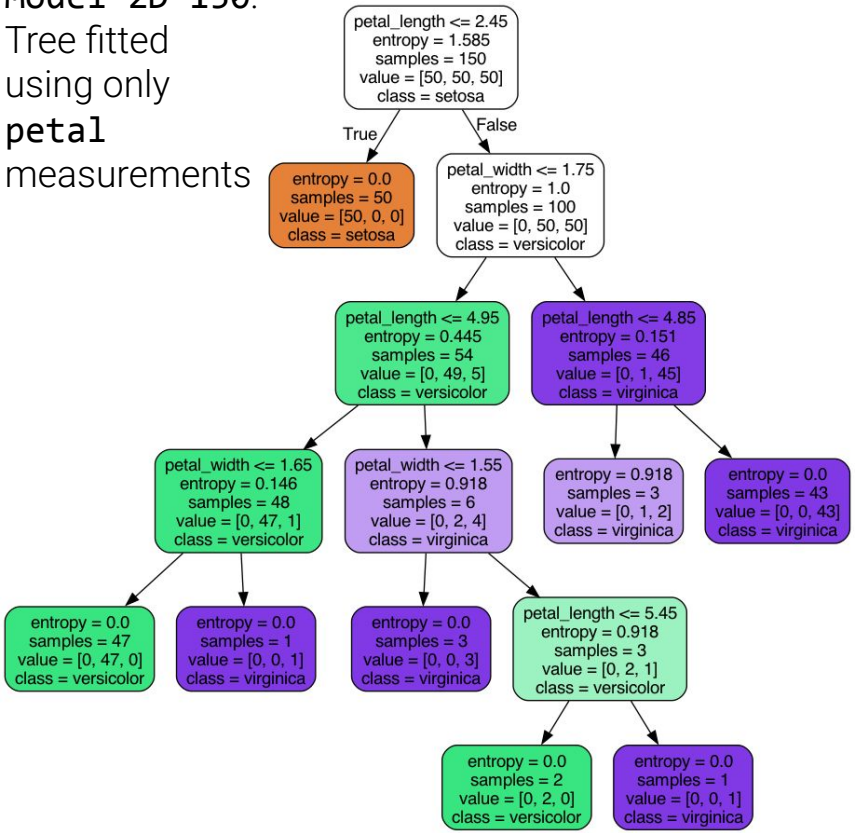
Cannot easily visualize Model 4D-110's decision boundaries because the prediction space is 4 dimensional.

- However, we can visualize the decision tree itself.

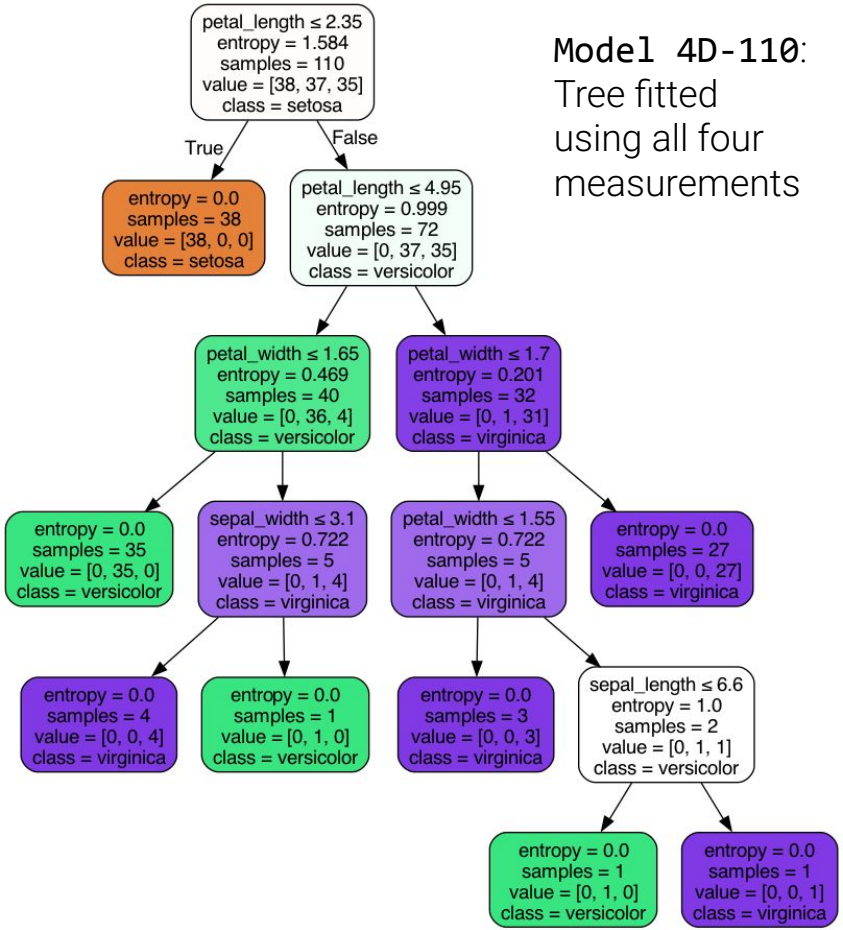
CC BY NC SA

Model 2D-150 vs. Model 4D-110 Tree Visualization

`Model 2D-150`: Tree fitted using only **`petal`** measurements

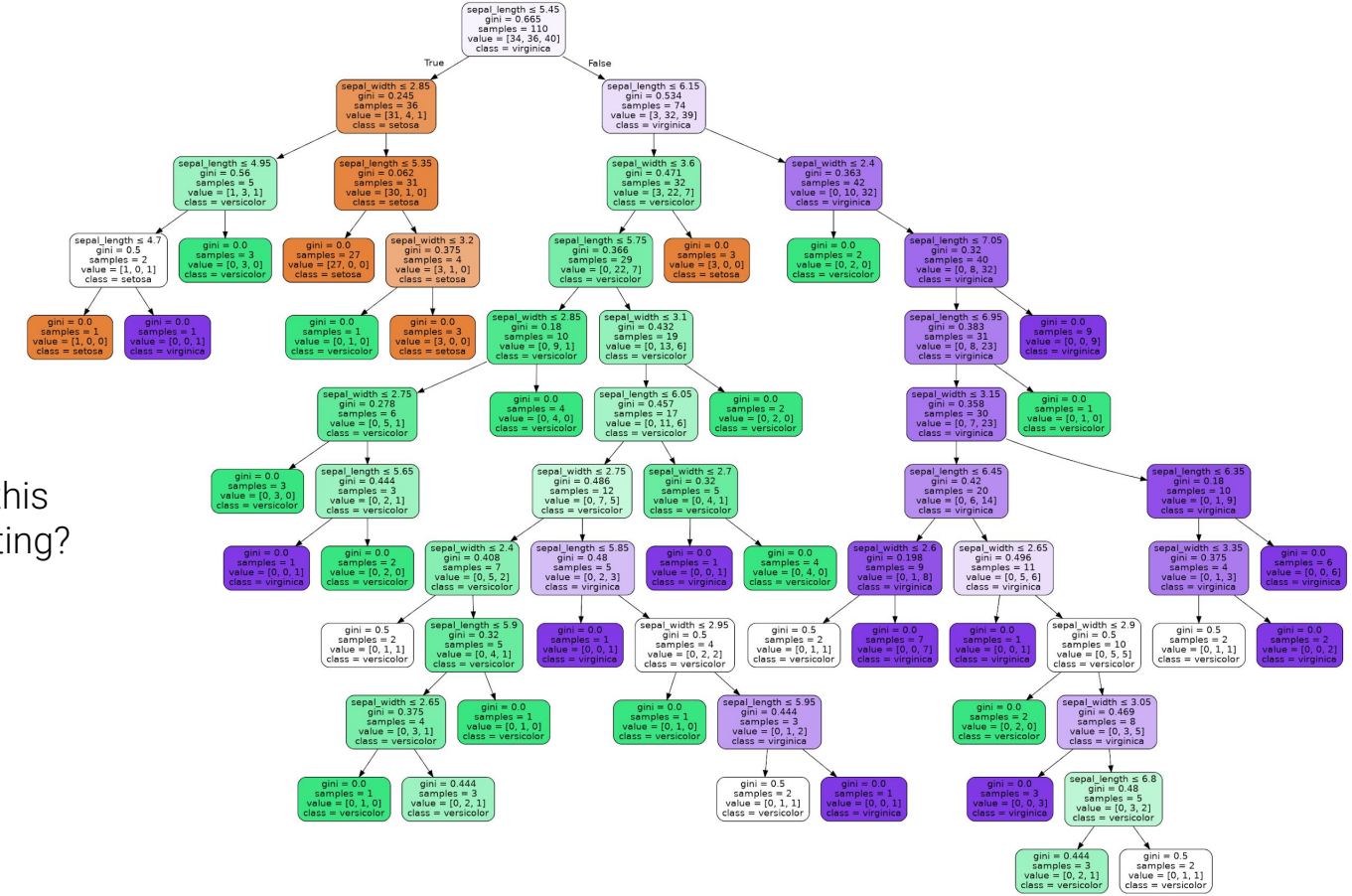


`Model 4D-110`: Tree fitted using all four measurements



CC BY NC SA

Sepal Decision Tree



OK… how did this end up overfitting?

CC BY NC SA

Evaluating Split Quality

Multiclass Classification

Decision Trees (conceptually)

- Creating Decision Trees in `sklearn`
- Evaluating Tree Accuracy
- Overfit Decision Tree Example

The Decision Tree Generation Algorithm

- **Evaluating Split Quality**
- Entropy
- Generating Trees Using Entropy

Avoiding Overfitting

- Restricting Decision Tree Complexity
- Random Forests

Summary and Context

CC BY NC SA

Sepal Decision Tree

In order to understand how to avoid overfitting, let's first discuss how decision trees are created from data.

Traditional decision tree generation algorithm:

- All of the data starts in the root node.
- Repeat until every node is either **pure** or **unsplittable**:
  - Pick the ***best feature*** x and ***best split value*** β, e.g. x = **"`petal_length`"**, β = 2.
  - *Split data into two nodes*, one where $x \leq \beta$, and one where $x > \beta$.

Pure? Unsplittable?

A node that has only samples from **one class** is called a "**pure**" node.

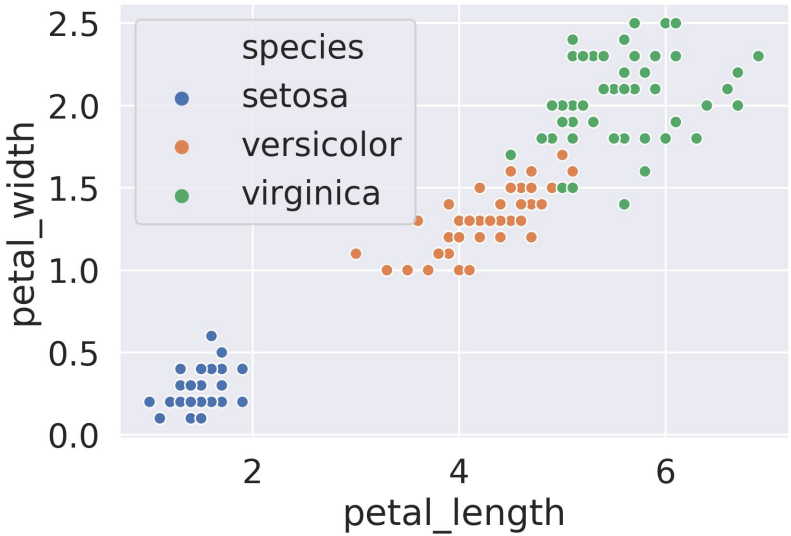
A node that has overlapping data points from different classes and thus that cannot be split is called "**unsplittable**".

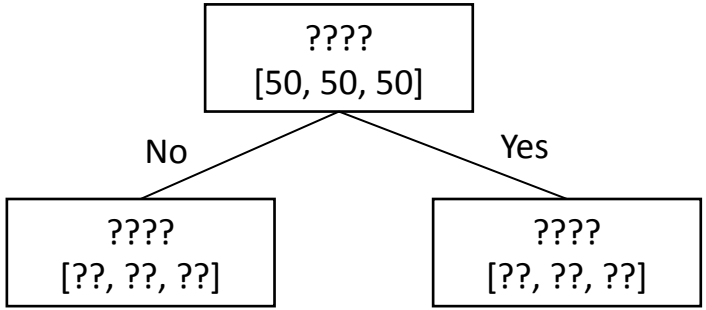
CC BY NC SA

Defining a Best Feature

Question: Which feature and split value is best?

- Equivalently: Which horizontal or vertical line do we want to draw?



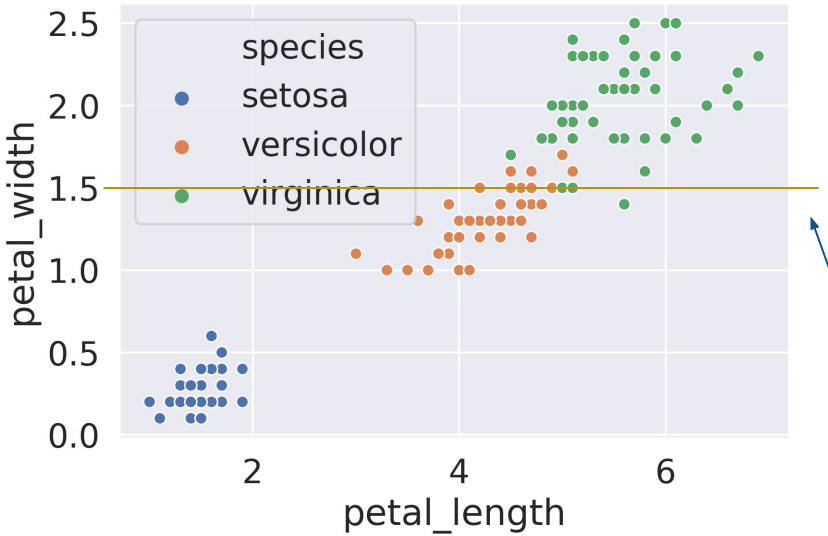


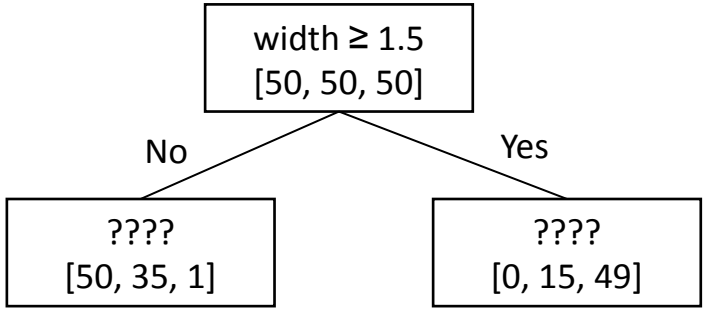
CC BY NC SA

Defining a Best Feature

Question: Which feature and split value is best?

- Equivalently: Which horizontal or vertical line do we want to draw?





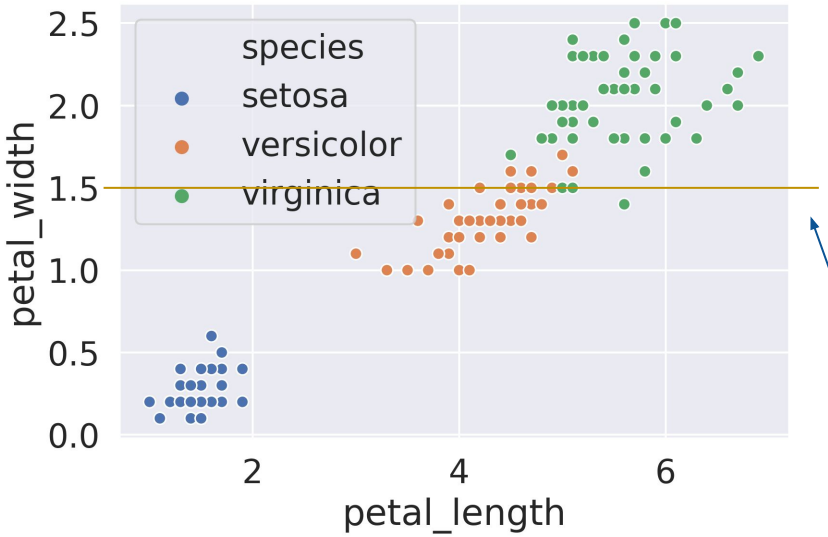
Is this good?

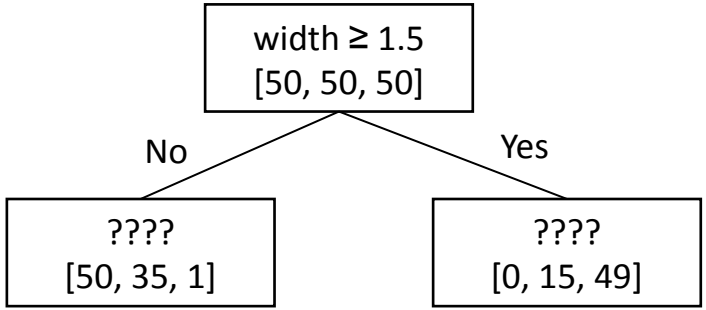
CC BY NC SA

Defining a Best Feature

Question: Which feature and split value is best?

- Equivalently: Which horizontal or vertical line do we want to draw?





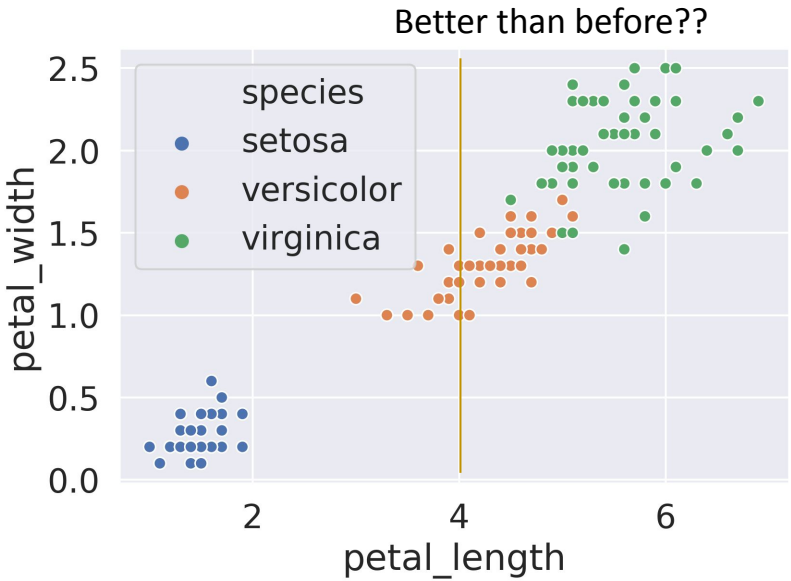
Is this good? It does help, but we could do better!

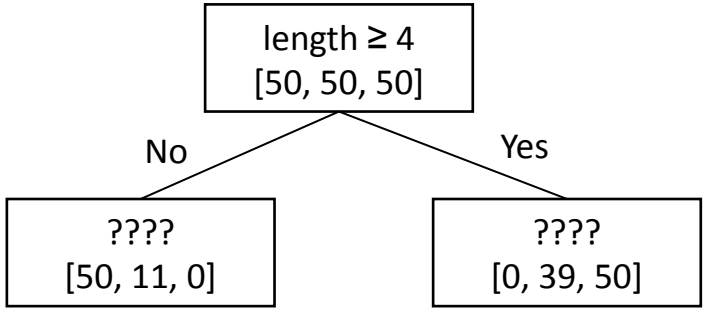
CC BY NC SA

Defining a Best Feature

Question: Which feature and split value is best?

- Equivalently: Which horizontal or vertical line do we want to draw?



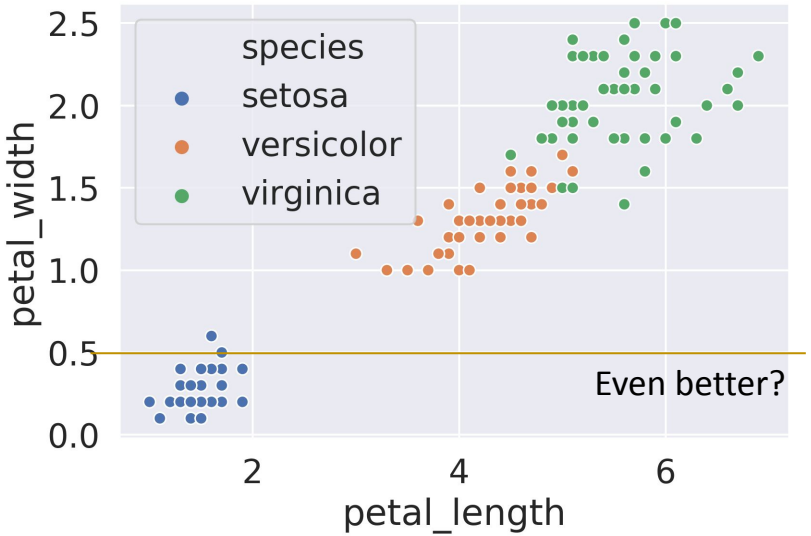


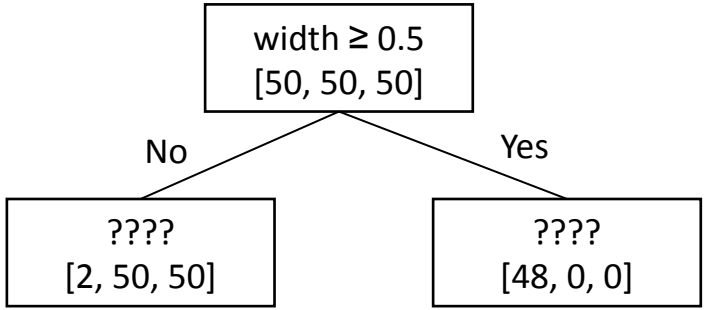
CC BY NC SA

Defining a Best Feature

Question: Which feature and split value is best?

- Equivalently: Which horizontal or vertical line do we want to draw?





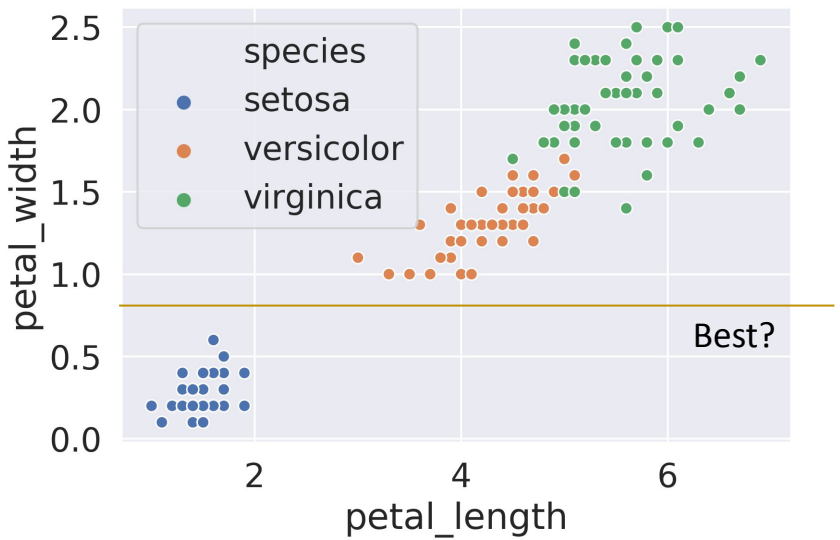
CC BY NC SA

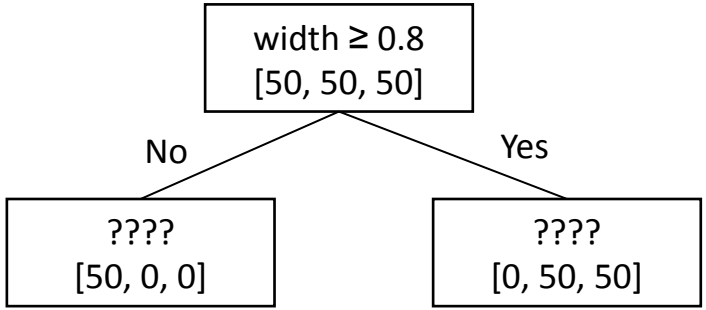
Defining a Best Feature

Question: Which feature and split value is best?

- Equivalently: Which horizontal or vertical line do we want to draw?

We need some sort of rigorous definition for a good split.





CC BY NC SA

Entropy

Multiclass Classification

Decision Trees (conceptually)

- Creating Decision Trees in `sklearn`
- Evaluating Tree Accuracy
- Overfit Decision Tree Example

The Decision Tree Generation Algorithm

- Evaluating Split Quality
- **Entropy**
- Generating Trees Using Entropy

Avoiding Overfitting

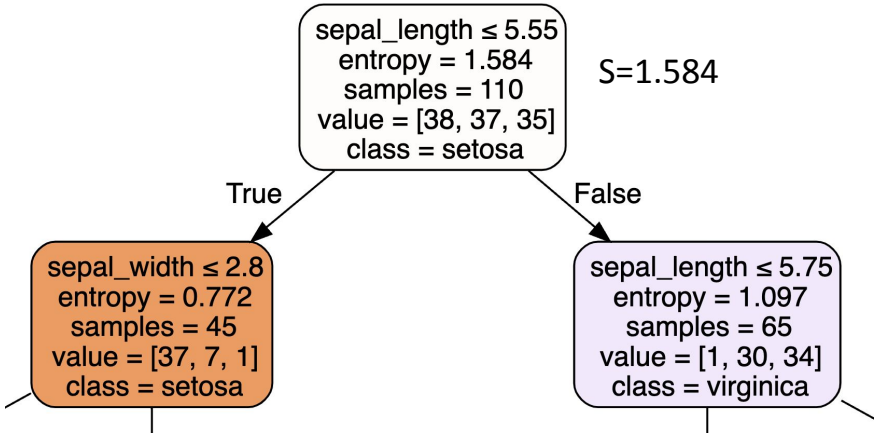
- Restricting Decision Tree Complexity
- Random Forests

Summary and Context

CC BY NC SA

Node Entropy

Let p_C be the proportion of data points in a node with label C.



For example, for the node at the top of the decision tree,

- p_0 = 38/110 = 0.35,
- p_1 = 37/110 = 0.34,
- p_2 = 35/110 = 0.32.

Define the **entropy** S of a node (in bits) as:

$$S = -\sum_c p_c \log_2 (p_c)$$

For example, S for the top node is:

$-0.35 \log_2 0.35 - 0.34 \log_2 0.34 - 0.32 \log_2 0.32 = 0.52 + 0.53 + 0.53 = 1.584$ bits

CC BY NC SA

Test Your Understanding

What are p_0, p_1, and p_2?

sepal_width < 2.8
entropy = ?
samples = 50
value = [0, 25, 25]
class = versicolor

What is the entropy of this node?

$$S = -\sum_c p_c \log_2 (p_c)$$

CC BY NC SA

Test Your Understanding

What are p_0, p_1, and p_2?

sepal_width < 2.8
entropy = ?
samples = 50
value = [0, 25, 25]
class = versicolor

What is the entropy of this node?

$$S = -\sum_c p_c \log_2 (p_c)$$

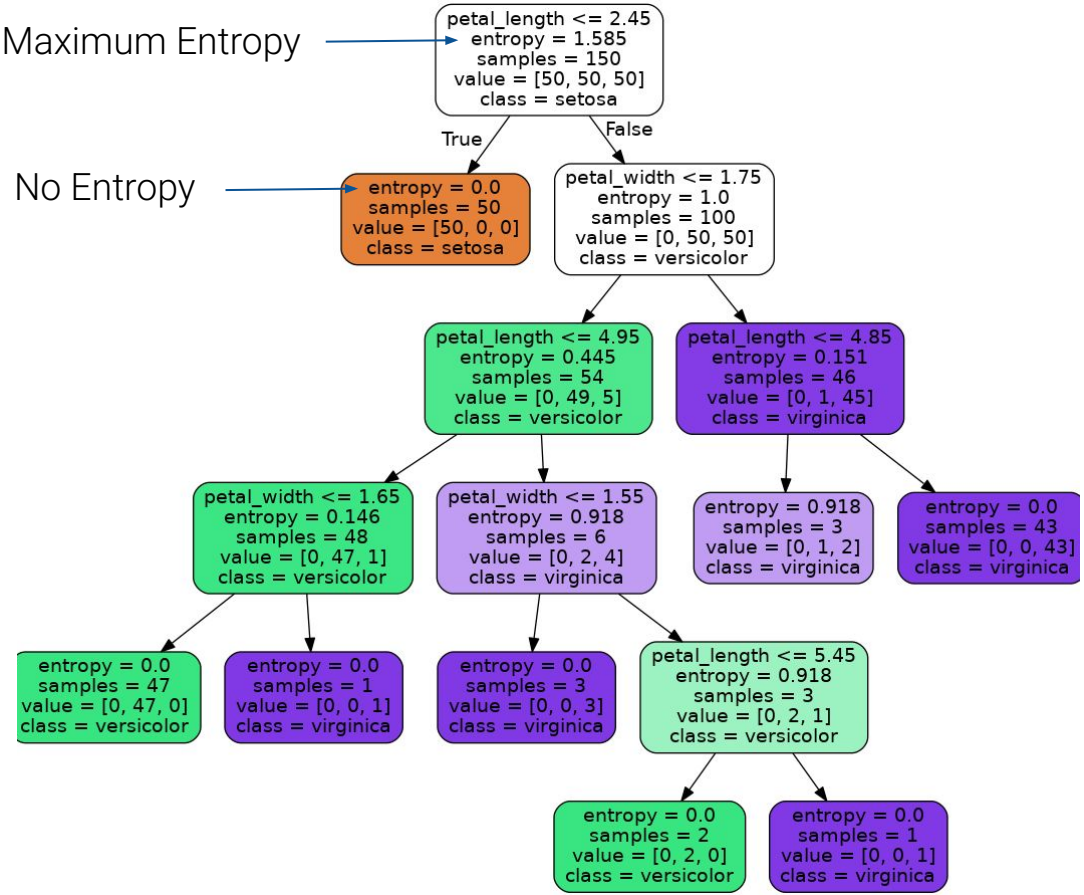
Can think of entropy as how unpredictable a node is. Low entropy means more predictable. High entropy means less predictable.

CC BY NC SA

Entropy of Nodes in Our First Decision Tree

Maximum Entropy

No Entropy



Recall: Entropy means uncertainty/surprise!

CC BY NC SA

Exploring Entropy

Observations about entropy:

- A node where all data are part of the same class has zero entropy.

  $-1 \log_2 1 = 0$ bits.

- A node where data are evenly split between two classes has entropy 1.

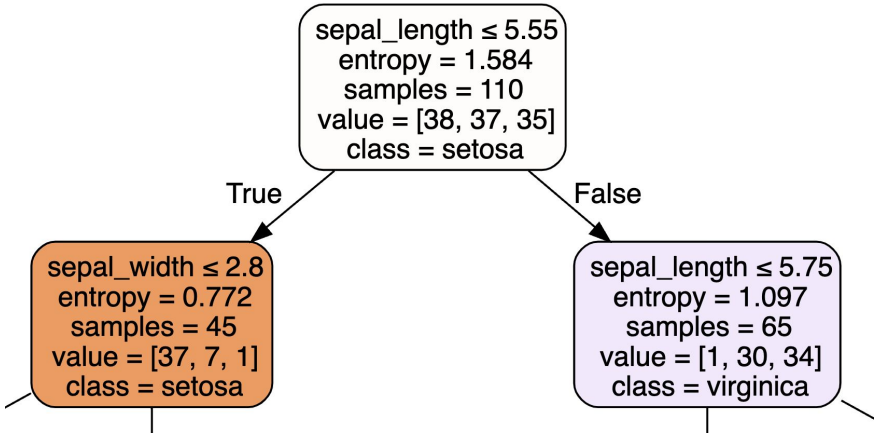
  $-0.5 \log_2 0.5 - 0.5 \log_2 0.5 = 1$ bit.

- A node where data are evenly split between 3 classes has entropy 1.58.

  $3 \times (-0.33 \log_2 0.33) = 1.58$ bits.

- A node where data are evenly split into C classes has entropy $\log_2 C$.

  $C \times (-1/C \log_2 1/C) = -\log_2 1/C = \log_2 C$



For more on entropy, see wiki or this youtube video.

CC BY NC SA

Generating Trees Using Entropy

Multiclass Classification

Decision Trees (conceptually)

- Creating Decision Trees in `sklearn`
- Evaluating Tree Accuracy
- Overfit Decision Tree Example

The Decision Tree Generation Algorithm

- Evaluating Split Quality
- Entropy
- **Generating Trees Using Entropy**

Avoiding Overfitting

- Restricting Decision Tree Complexity
- Random Forests

Summary and Context

CC BY NC SA

Weighted Entropy

We can use **Weighted Entropy** as a loss function in helping us decide which split to take

Suppose a given split results in two nodes X and Y with N_1 and N_2 total samples each. The loss of that split is given by:

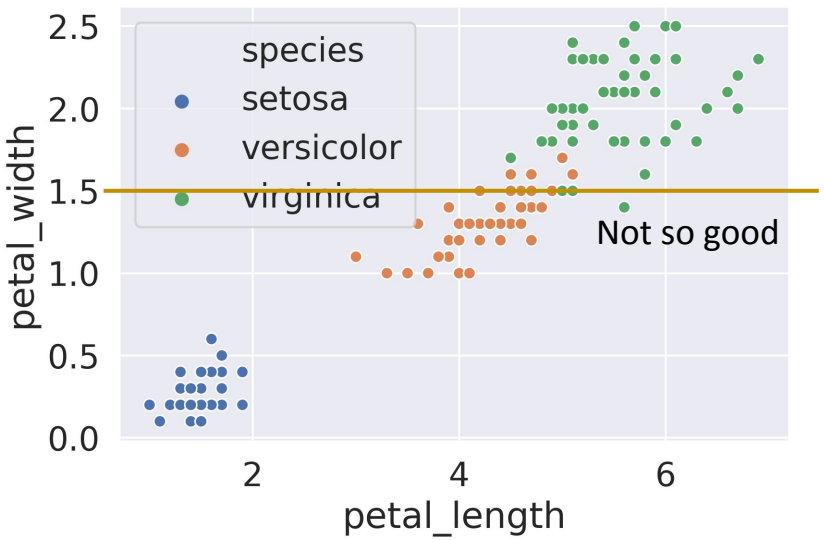
$$L = \frac{N_1 S(X) + N_2 S(Y)}{N_1 + N_2}$$

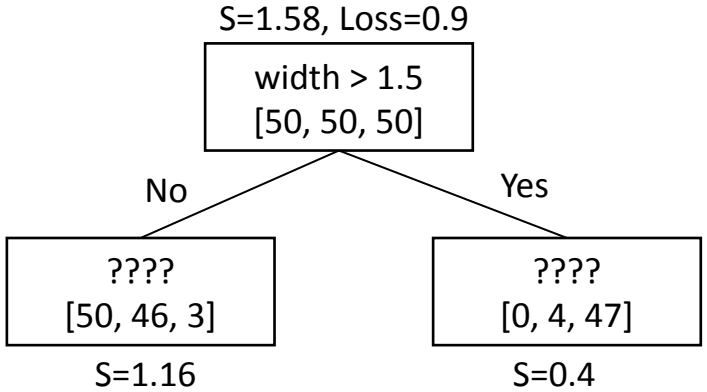
CC BY NC SA

Defining a Best Feature

Split choice #1: width > 1.5. Compute entropy of child nodes:

- entropy([50, 46, 3]) = 1.16
- entropy([4, 47]) = 0.4
- Weighted average: 99/150 × 1.16 + 51/150 × 0.4 = **0.9**



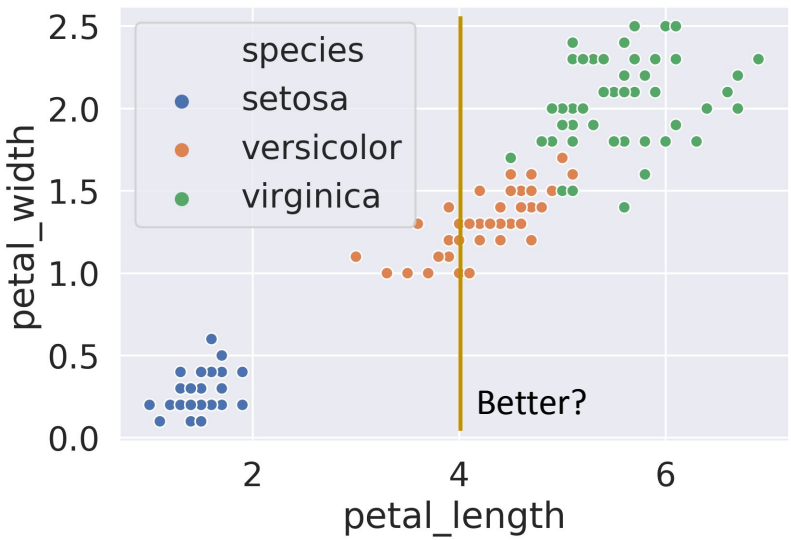


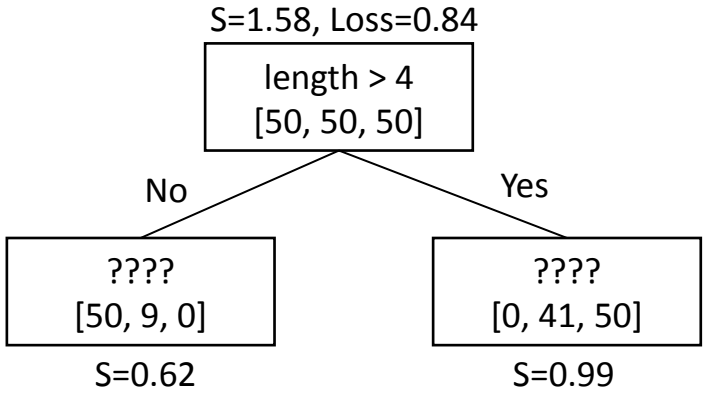
CC BY NC SA

Defining a Best Feature

Split choice #2: length > 4. Compute entropy of child nodes:

- entropy([50, 9]) = 0.62
- entropy([41, 50]) = 0.99
- Weighted Average: **0.84**: Better than split choice #1!



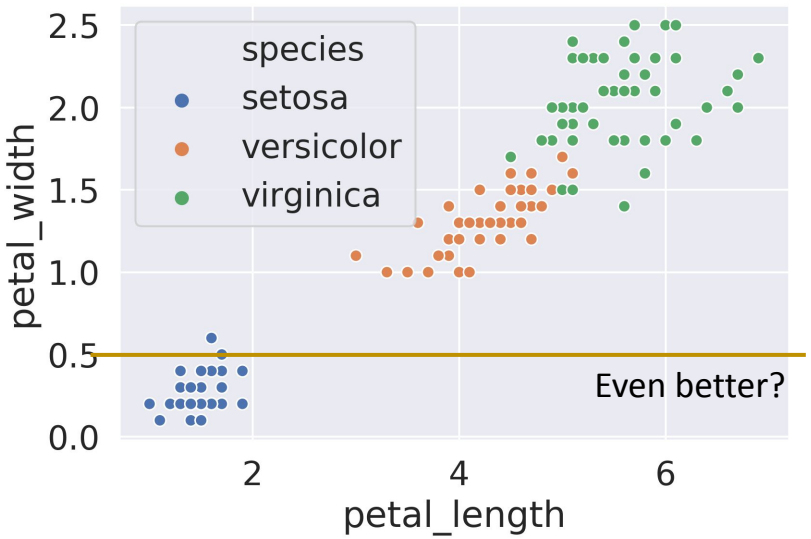


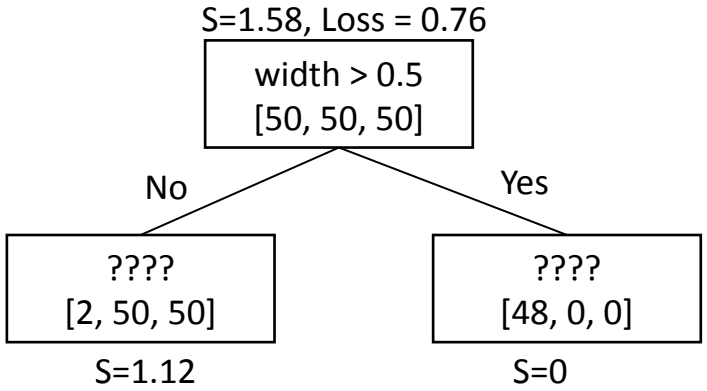
CC BY NC SA

Defining a Best Feature

Split choice #3: width > 0.5. Compute entropy of child nodes:

- entropy([2, 50, 50]) = 1.12
- entropy([48]) = 0
- Weighted average: **0.76**: Lower than split choice #2!



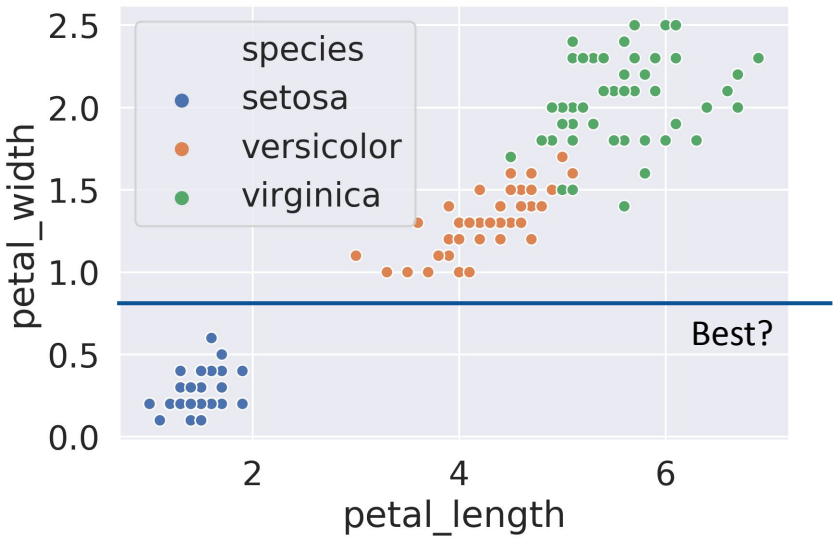


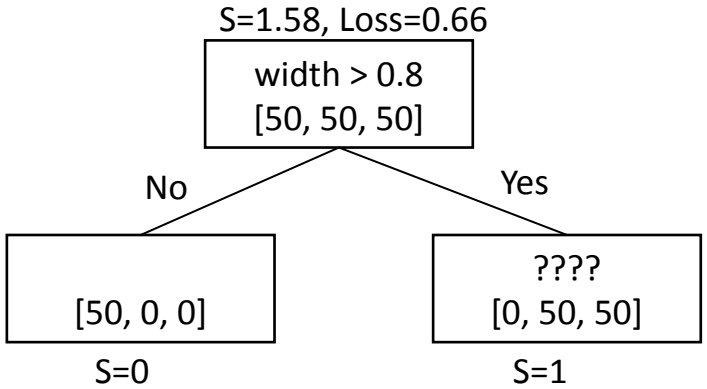
CC BY NC SA

Defining a Best Feature

Split choice #4: width > 0.9. Compute entropy of child nodes:

- entropy([50, 50]) = 1
- entropy([50]) = 0
- Weighted average: **0.66**: Lower than split choice #3!





CC BY NC SA

Decision Tree Generation

Decision tree generation algorithm:

- All of the data starts in the root node
- Repeat until every node is either **pure** or **unsplittable**:
  - Pick the ***best feature*** x and ***best split value*** β such that the **weighted entropy** of the resulting split is minimized, e.g. x = `petal_width`, β = 0.8 has loss 0.66
  - Split data into two nodes, one where $x \leq \beta$, and one where $x > \beta$

Notes: A node that has only one type is called a "**pure**" node. A node that has duplicate data that cannot be split is called "**unsplittable**"

Let's now turn our attention to avoiding overfitting

CC BY NC SA

Restricting Tree Complexity

Multiclass Classification

Decision Trees (conceptually)

- Creating Decision Trees in `sklearn`
- Evaluating Tree Accuracy
- Overfit Decision Tree Example

The Decision Tree Generation Algorithm

- Evaluating Split Quality
- Entropy
- Generating Trees Using Entropy

Avoiding Overfitting

- **Restricting Decision Tree Complexity**
- Random Forests

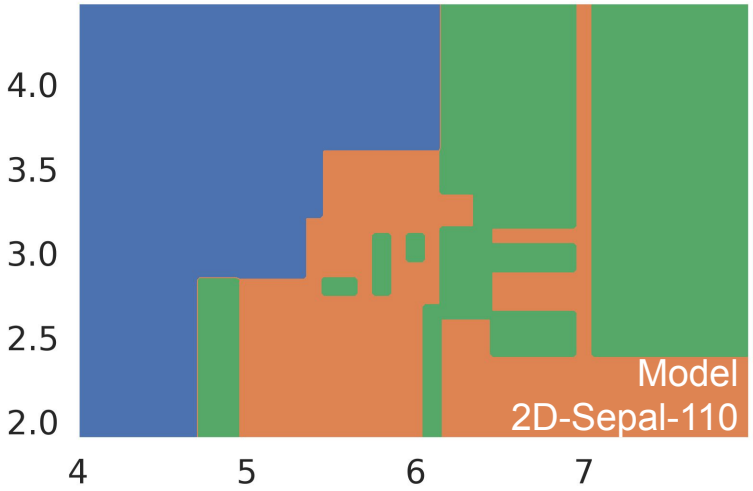
Summary and Context

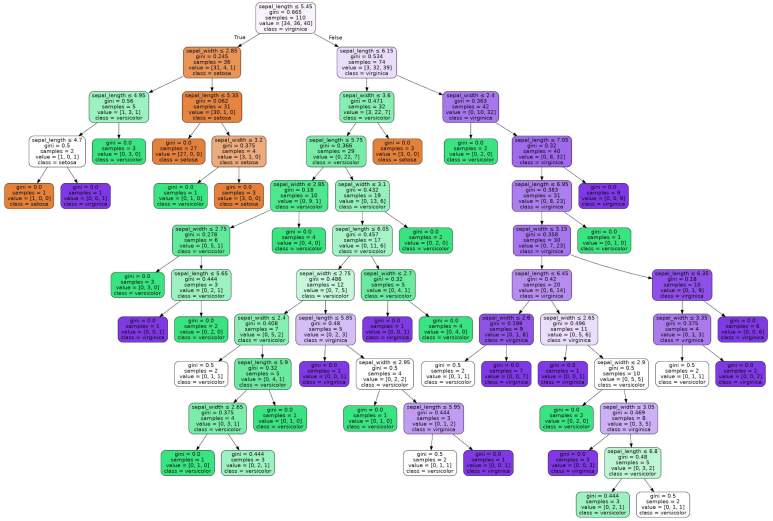
CC BY NC SA

Overfitting and Our Algorithm

A “fully grown” decision tree built with our algorithm runs the risk of overfitting.

- In linear models, we introduced the notion of “regularization”.
- Regularization doesn’t make sense in the decision tree context. There’s no weights to be penalized. Have to come up with a new idea.



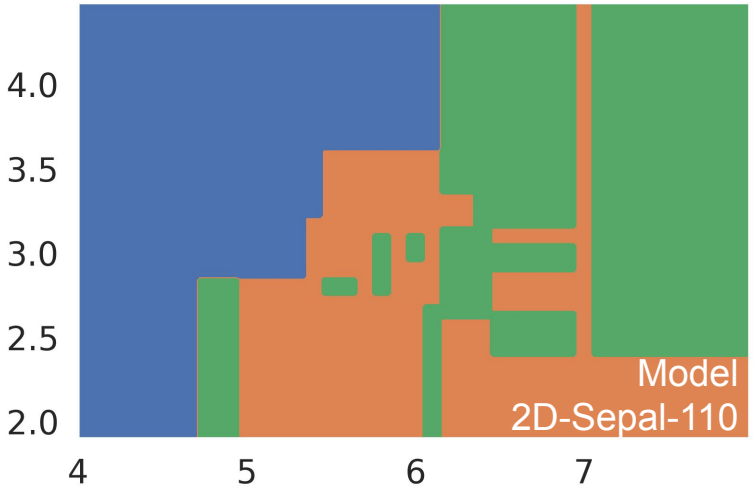


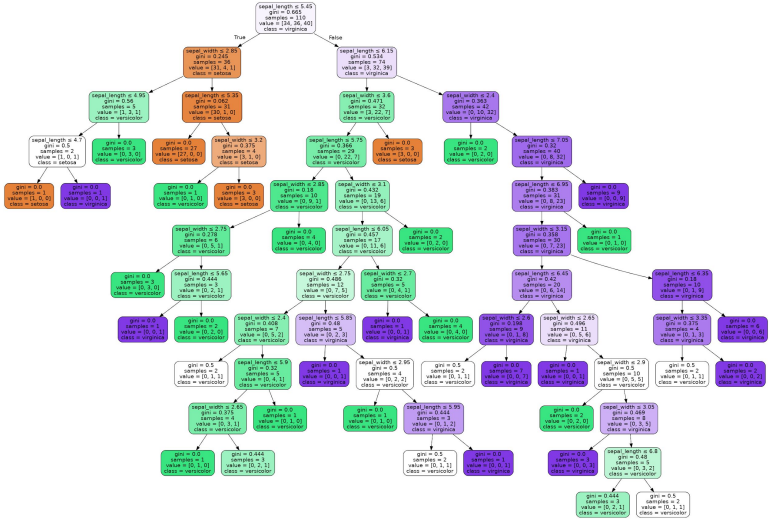
CC BY NC SA

Overfitting and Our Algorithm

A “fully grown” decision tree built with our algorithm runs the risk of overfitting.

One idea to avoid overfitting: Don’t allow fully grown trees.





CC BY NC SA

Approach 1: Preventing Growth

Approach 1: Set one or more special rules to prevent growth.

Examples:

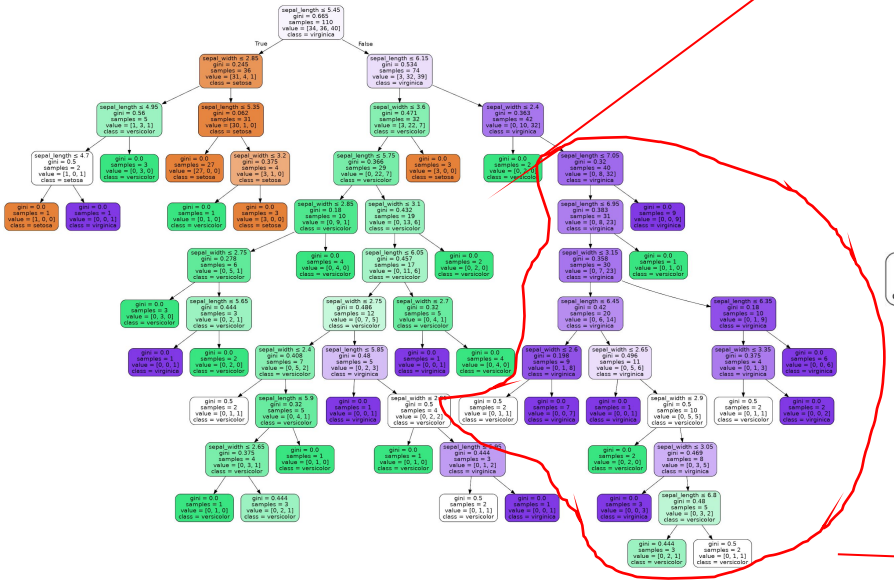
- Don't split nodes with < 1% of the samples.
  - In `sklearn`, this hyperparameter is called "`min_samples_split`".
- Don't allow nodes to be more than 7 levels deep in the tree.
  - In `sklearn`, this hyperparameter is called "`max_depth`".

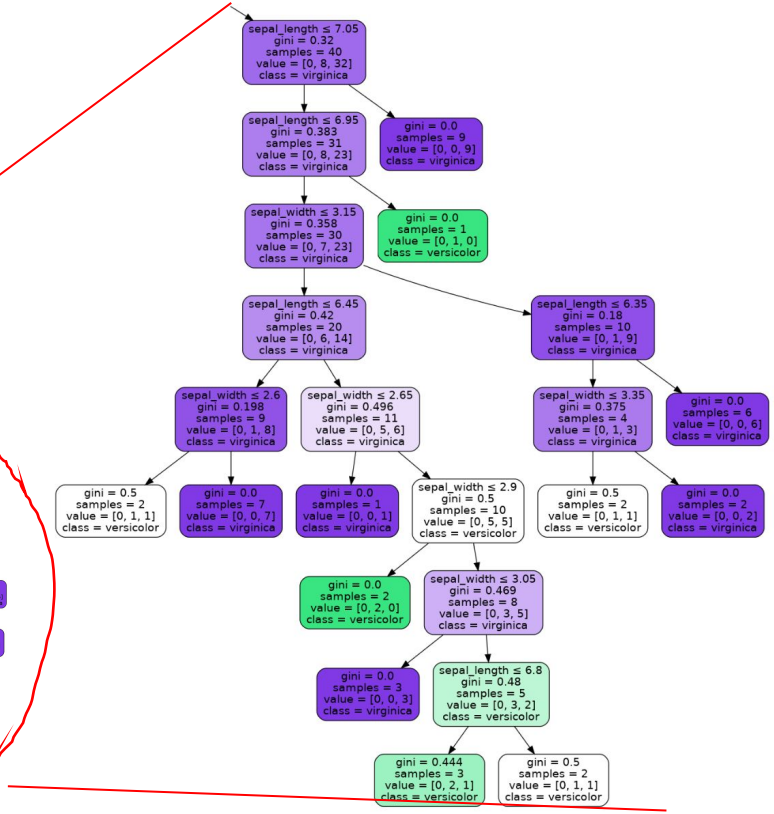
CC BY NC SA

Approach 2: Pruning

Approach 2: Let tree fully grow, then cut off less useful branches of the tree. For example, consider the highlighted branch below:

- Many rules that affect few points.



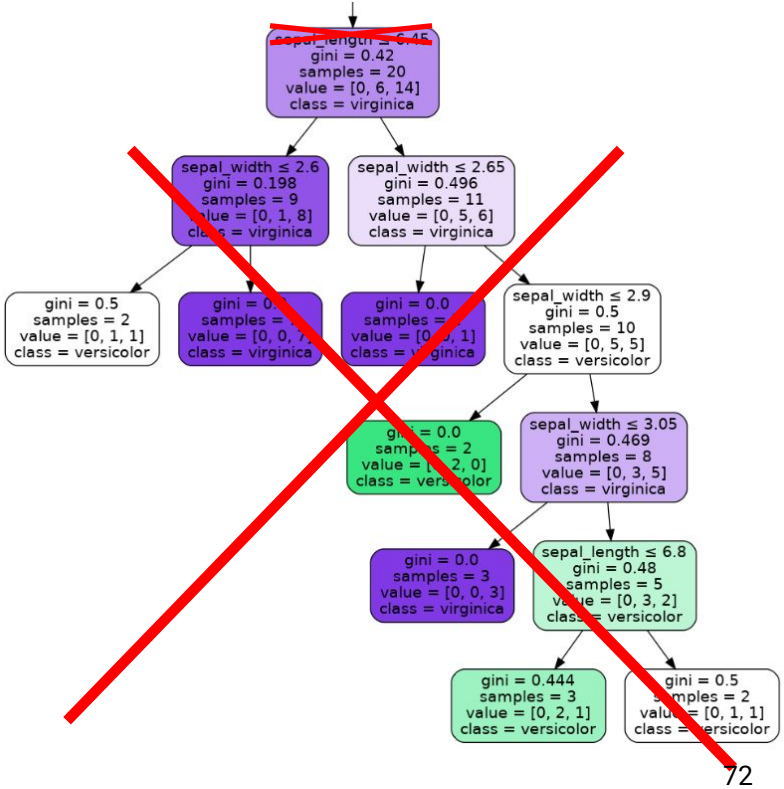


CC BY NC SA

Specific Pruning Example

One way to prune:

- Before creating the tree, set aside a validation set.
- If replacing a node by its most common prediction has no impact on the validation error, then don't split that node.



CC BY NC SA

Participation Questions

CC BY NC SA

Q1

Which form of regularization can *not* be used with decision trees?

A. Preventing growth

B. Restricting growth

C. L2 Norm

D. Removing features at random

CC BY NC SA

Q2

Entropy can be seen as:

A. The inevitable heat death of the universe

B. A measure of the variability of a node

C. A measure of the complexity of a node

D. The average of the classes within a node

CC BY NC SA

Q3

Which of the following **is** a valid split in a decision tree?

A. petal_length < 2 and petal_width ≥ 1.5

B. petal_length ≥ 2.5

C. petal_length < 0

D. 2 < petal_length < 4

CC BY NC SA

Q4

Which splits does a decision tree check before choosing the optimal split?

A. All possible values of all features

B. All values that are either equal to an observation's value, or halfway in between, for all features.

C. All values that are exactly equal to an observation's value for all features

D. All possible rectangular regions in the plot of one feature versus another

CC BY NC SA

Random Forests

Multiclass Classification

Decision Trees (conceptually)

- Creating Decision Trees in `sklearn`
- Evaluating Tree Accuracy
- Overfit Decision Tree Example

The Decision Tree Generation Algorithm

- Evaluating Split Quality
- Entropy
- Generating Trees Using Entropy

Avoiding Overfitting

- Restricting Decision Tree Complexity
- **Random Forests**

Summary and Context

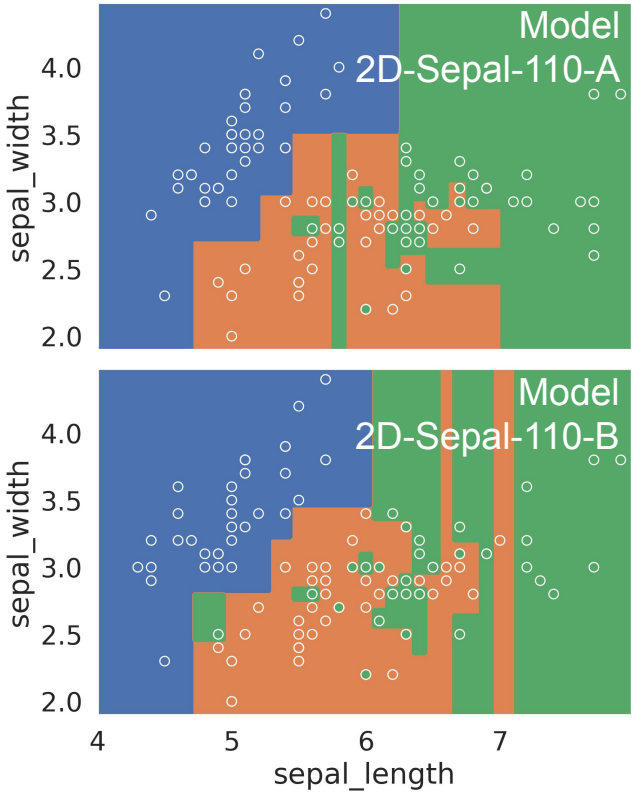
CC BY NC SA

Random Forests: Harnessing Variance

As we've seen, fully-grown decision trees will almost always overfit data

- Low model bias, high model variance.
- In other words, small changes in dataset will result in very different decision tree.
- **Example**: Two models on the right trained on different subsets of the same data.

Random Forest Idea: Build many decision trees and have them vote.



CC BY NC SA

Random Forests: Harnessing Variance

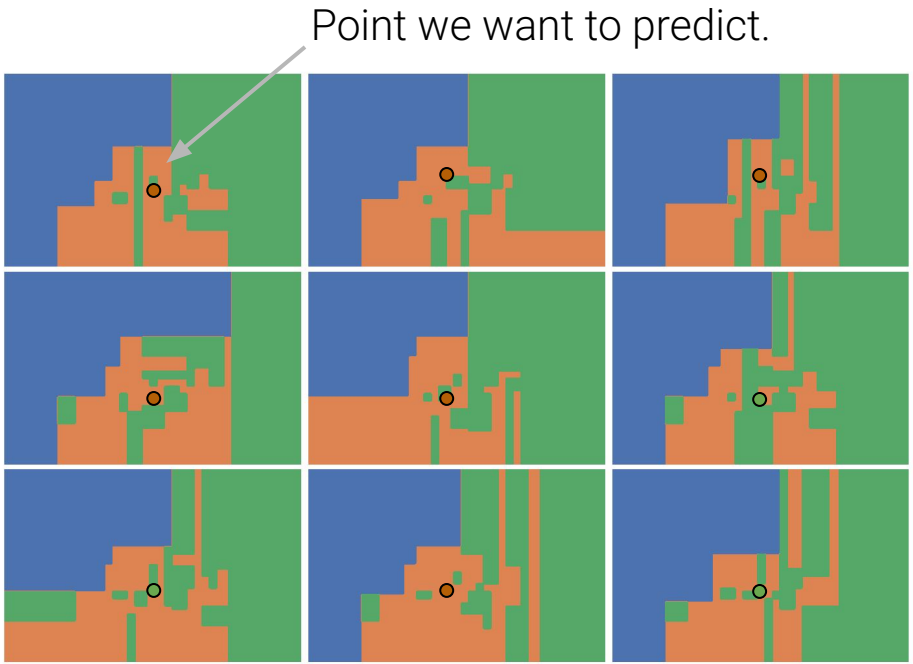
What do we mean by vote?

- For a given x/y, use whichever prediction is most popular.

Consider example at right with 9 models.

- 6 votes orange, 3 votes green.
- Random forest prediction is orange.

This kind of methods is called **ensemble learning**.



CC BY NC SA

Building Many Trees

Big fundamental problem: We only have one training set.

How can we build many trees using one training set?

CC BY NC SA

How can we build many trees using one training set?

BY NC SA

Bagging

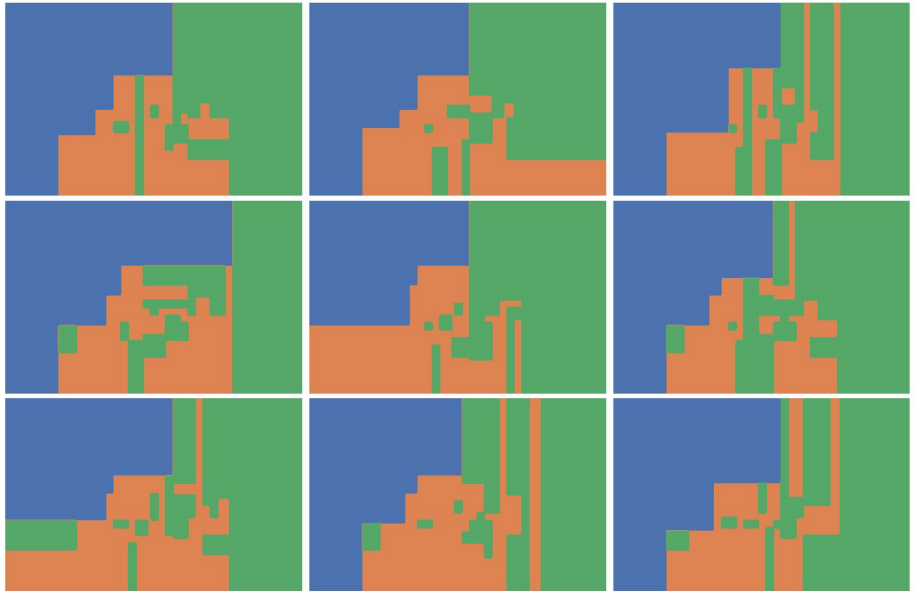
Bagging: Short for Bootstrap AGGregatING.

- Generate bootstrap resamples of training data.
- Fit one model for each resample.
- Final model = average predictions of each small model.
- Invented by Leo Breiman in 1994 (Berkeley Statistics!).

CC BY NC SA

Random Forests

- Bagging often isn't enough to reduce model variance!
  - Decision trees often look very similar to each other and make similar predictions.
  - Ensemble will still have low bias and high model variance.
  - E.g., one strong feature always used for first split.



CC BY NC SA

Random Forests

- Bagging often isn't enough to reduce model variance!
  - Decision trees often look very similar to each other and make similar predictions.
  - Ensemble will still have low bias and high model variance.
  - E.g., one strong feature always used for first split.

- **Idea:** Only use a random subset of ***m*** features at each split.
  - Usually $m = \sqrt{p}$ for decision trees used for classification.
  - Here p is the number of features.
- Algorithm creates individual trees, each overfit in a different way.
  - The hope is that the overall forest has low variance.

CC BY NC SA

Random Forests Algorithm

- *Bootstrap training* data T times. For each resample, fit a decision tree by doing the following:
  - Start with data in one node. Until all nodes **pure** or **unsplittable**:
    - Pick an impure node.
    - *Pick a random subset of m features.* Pick the ***best feature*** x and ***best split value*** β such that the **weighted entropy** of the resulting split is minimized
    - Split data into two nodes, one where $x \leq \beta$, and one where $x > \beta$
- To predict, ask the T decision trees for their predictions and take majority vote.

This approach has two hyperparameters T and m.

CC BY NC SA

Avoiding Overfitting with Heuristics

We've seen many approaches to avoid overfitting decision trees.

- Preventing growth.
- Pruning.
- Random forests.

These ideas are generally "heuristic"

- Not provably best or mathematically optimal.
- Instead, they are just ideas that somebody thought sounded good, implemented, then found to work in practice acceptably well.

CC BY NC SA

Summary and Context

Multiclass Classification

Decision Trees (conceptually)

- Creating Decision Trees in `sklearn`
- Evaluating Tree Accuracy
- Overfit Decision Tree Example

The Decision Tree Generation Algorithm

- Evaluating Split Quality
- Entropy
- Generating Trees Using Entropy

Avoiding Overfitting

- Restricting Decision Tree Complexity
- Random Forests

Summary and Context

CC BY NC SA

Decision Tree Generation

Decision tree generation algorithm:

- All of the data starts in the root node
- Repeat until every node is either **pure** or **unsplittable**:
  - Pick the ***best feature*** x and ***best split value*** β such that the **weighted entropy** of the resulting split is minimized, e.g. x = `petal_width`, β = 0.8 has loss 0.66
  - Split data into two nodes, one where $x \leq \beta$, and one where $x > \beta$

Notes: A node that has only one type is called a "**pure**" node. A node that has duplicate data that cannot be split is called "**unsplittable**"

CC BY NC SA

Why Decision Trees?

Decision Trees present an alternative philosophy to logistic regression.

- Instead of a weighted sum of features, we use a series of yes/no questions.

Some advantages of a decision tree:

- Able to capture nonlinear relationships (at risk of overfitting).
- Output is more interpretable: You know exactly how the tree made a decision.
  - Example: Imagine trying to make an argument in a court case.
  - This is not true of random forests.

Some disadvantages:

- Care needs to be taken to avoid overfitting.
- Not as mathematically nice (e.g. Ridge Regression has a simple closed form solution).

If all you care about is accuracy (or precision) (or recall), use **cross validation** to pick between the two modeling philosophies.

CC BY NC SA

Random Forests Algorithm

- *Bootstrap training* data T times. For each resample, fit a decision tree by doing the following:
  - Start with data in one node. Until all nodes **pure** or **unsplittable**:
    - Pick an impure node.
    - *Pick a random subset of m features.* Pick the ***best feature*** x and ***best split value*** β such that the **weighted entropy** of the resulting split is minimized
    - Split data into two nodes, one where $x \leq \beta$, and one where $x > \beta$
- To predict, ask the T decision trees for their predictions and take majority vote.

This approach has two hyperparameters T and m.

CC BY NC SA

Why Random Forests?

- Versatile: does both regression and classification.
- Invariant to feature scaling and translation.
- Automatic feature selection.
- Nonlinear decision boundaries without complicated feature engineering.
- Doesn't overfit as often as other nonlinear models (e.g. polynomial features).
- Example of **ensemble method**: combine the knowledge of many simple models to make a sophisticated model.
- Example of using **bootstrap** to reduce model variance.

CC BY NC SA